

Entwicklung einer Überwachungs- und Diagnose-Applikation für das Kommunikationsprotokoll zwischen den Teilmaschinen im Linienverbund

Projektarbeit T4_2000

Studiengang Elektrotechnik und Informationstechnik

Studienrichtung Automation

Duale Hochschule Baden-Württemberg Ravensburg, Campus Friedrichshafen

von

Fabian Wiedemann

Abgabedatum:	07.09.2026
Bearbeitungszeitraum:	30.03.2026 - 07.09.2026
Matrikelnummer:	6415375
Kurs:	TEA24
Dualer Partner:	MULTIVAC Sepp Haggenmüller SE & Co.KG
Betreuer:	M. Eng. Dipl.-Ing.(FH) Markus Bachmann

Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe und diese Arbeit bei keiner anderen Prüfung mit gleichem oder vergleichbarem Inhalt vorgelegt habe und diese bislang nicht veröffentlicht wurde.

Musterstadt, den 6. August 2026

Fabian Wiedemann

Kurzfassung

Die Verpackungsmaschinen bei der Firma MULTIVAC bestehen aus mehreren miteinander vernetzten Maschinen, die kontinuierlich Zustände, Ereignisse und Prozessdaten austauschen müssen, damit ein reibungsloses Zusammenspiel zwischen ihnen funktioniert. Dieser Austausch erfolgt über das Nachrichtenprotokoll MQTT (Message Queuing Telemetry Transport). Die Analyse der dabei entstehenden Nachrichtenströme gestaltet sich insbesondere bei komplexen Anlagen und hohen Nachrichtenvolumina schwierig. Bereits bestehende Werkzeuge ermöglichen zwar die Anzeige von Nachrichten, wodurch eine systematische Fehlersuche und Diagnose aber nur eingeschränkt möglich sind und ein umfangreiches Expertenwissen voraussetzen.

Ziel dieser Arbeit war die Entwicklung einer Überwachungs- und Diagnose-Applikation zur Analyse der MQTT-basierten Maschinenkommunikation in Verpackungslinien. Die Anwendung soll Nachrichtenströme transparent darstellen, dauerhaft speichern und eine effiziente Analyse großer Datenmengen ermöglichen. Zusätzlich sollen Kommunikations- und Prozessfehler automatisch erkannt sowie Funktionen zur Teilnehmerüberwachung und Nachrichtensimulation bereitgestellt werden.

Zunächst wurden bestehende Monitoring-Werkzeuge hinsichtlich ihres Funktionsumfangs sowie ihrer Eignung für den geplanten Anwendungsfall untersucht. Anschließend wurde ein prototypisches System in Python entwickelt, um technische Anforderungen zu identifizieren und die Machbarkeit der Umsetzung zu untersuchen. Darauf aufbauend wurde eine Softwarearchitektur auf Basis des MVVM-Entwurfsmusters konzipiert. Die finale Umsetzung erfolgte in der Programmiersprache C# unter Verwendung von WPF für die Benutzeroberfläche, MQTTnet für die MQTT-Kommunikation sowie SQLite zur persistenten Datenspeicherung. Die Verarbeitung der Nachrichten erfolgt ereignisgesteuert und asynchron, um auch hohe Nachrichtenraten effizient verarbeiten zu können.

Im Rahmen dieser Arbeit wurde eine funktionsfähige Monitoring- und Diagnose-Applikation entwickelt. Die Anwendung ermöglicht die dauerhafte Speicherung und Volltextsuche von MQTT-Nachrichten, die Überwachung aller Kommunikationsteilnehmer sowie die Analyse von Prozessabläufen wie Linienstart, Linienstopp und Artikelwechsel. Zusätzlich wurden Mechanismen zur au-

tomatischen Erkennung von Struktur- und Prozessfehlern implementiert. Darüber hinaus wurden Funktionen zum manuellen Versand von MQTT-Nachrichten bereitgestellt. Mithilfe der SQLite-Datenbank und eines Paging-Konzepts können auch große Mengen an Kommunikationsdaten effizient verwaltet und analysiert werden. Die Architektur ist modular aufgebaut und kann zukünftig um weitere Analyse- und Diagnosefunktionen erweitert werden.

Abstract

Packaging lines produced at the company MULTIVAC consist of multiple interconnected machines that continuously exchange information regarding machine states, events and process data to ensure reliable and coordinated operation. This communication is realized using the Message Queuing Telemetry Transport (MQTT) protocol. The analysis of the resulting message streams becomes increasingly challenging in complex production lines and under high message volumes. Although existing monitoring tools provide the capability to display sent MQTT messages, systematic fault diagnosis and communication analyses are only possible to a limited extent and often require expert knowledge.

The objective of this thesis was to develop a working monitoring and diagnostic application for MQTT-based communication. The application is intended to provide a transparent visualization of message streams, support the persistent storage of communication data, and enable the efficient analysis of large message volumes. Additionally, the application should automatically detect errors in communication and processes and provide functionality for participant monitoring and message simulation.

Initially, existing tools for monitoring and displaying MQTT messages were evaluated to determine whether they fulfilled the requirements of the intended use case. Subsequently, a prototype was developed in Python to assess the technical feasibility of the application and to identify relevant requirements and challenges. Based on the insights gained, a software architecture following the Model-View-ViewModel (MVVM) pattern was designed. The final application was implemented in C# using Windows Presentation Foundation (WPF) for the graphical user interface, MQTTnet for MQTT communication, and SQLite for persistent data storage. Message processing is performed in an event-driven and asynchronous manner, enabling the efficient handling of high message rates and large volumes of communication data.

As a result of this project, a fully functional monitoring and diagnostic application was developed. The application enables the persistent storage and retrieval of MQTT messages through integrated

search and filtering mechanisms. Furthermore, it provides functionality for monitoring communication participants and analysing production-line processes, including line start, line stop, article change and continuous article change. In addition, mechanisms for the automatic detection of structural and process-related errors were implemented, alongside functionality for the manual simulation and transmission of MQTT messages. By utilizing an SQLite database in combination with a paging concept, the application is capable of efficiently managing and analysing large volumes of communication data. The software architecture was designed in a modular manner, allowing future monitoring and analysis features to be integrated with minimal effort. This ensures both maintainability and extensibility of the application for future requirements.

Inhaltsverzeichnis

1	Einleitung	1
2	Unternehmens- und Projektumfeld	2
2.1	Vorstellung des Praxispartners	2
2.2	Maschinenkommunikation in Produktionslinien	2
2.3	Herausforderungen bei der Analyse von Kommunikationsabläufen	3
3	Grundlagen	5
3.1	Grundprinzip MQTT	5
3.1.1	Publisher	5
3.1.2	Subscriber	6
3.1.3	Broker	6
3.1.4	Topics als Adressierungsmechanismus	7
3.2	Architektur der MQTT-Nachrichten	7
3.2.1	Aufbau eines MQTT-Control-Packets	7
3.2.2	Datensicherheit bei MQTT	8
3.3	Quality of Service	9
3.3.1	Allgemeine Definition	9
3.3.2	Übersicht der Quality-of-Service-Level	9
3.4	Vorteile von MQTT in der Automatisierung	10
3.5	.Net als Laufzeitplattform	11
3.6	XAML und Deklarative Benutzeroberflächen	11
3.7	Model-View-ViewModel als Architekturstruktur	12
4	Ausgangssituation	14
4.1	Monitoring über MQTT.fx	14
4.1.1	Funktionalität von MQTT.fx	14
4.1.2	Gründe für und gegen den Einsatz von MQTT.fx	16
4.2	Monitoring über MQTT Explorer	16

4.3	Möglichkeit der Nutzung und Erweiterung bereits existenter Applikationen	18
4.3.1	Funktionsweise von MQTTMultimeter	18
4.3.2	Technische Realisierung von MQTTMultimeter	20
4.3.3	Gründe gegen die Erweiterung von MQTTMultimeter	21
5	Technische Rahmenbedingungen	23
5.1	Anforderungen an die Programmiersprache	23
5.1.1	Eignung der Sprache C# für die Umsetzung	23
5.1.2	MQTT-Client-Bibliothek	24
5.1.3	Datenbank-Bibliothek	24
5.1.4	Graphical User Interface Framework	26
5.2	Anforderungen an die Test- und Entwicklungsumgebung	28
5.2.1	Entwicklungsumgebung	28
5.2.2	Testumgebung	29
6	Technisches Konzept	30
6.1	Anforderungen an die Applikation	30
6.1.1	Nachrichtendarstellung und Filterung	30
6.1.2	Automatische Fehlererkennung	30
6.1.3	Überwachung der Automatisierungs-Abläufe	31
6.1.4	Simulationsmöglichkeiten	31
6.1.5	Anforderungen an die Performance	32
6.1.6	Erweiterbarkeit und Zukunftssicherheit	32
6.2	Prototypische Umsetzung und Erkenntnisse aus der Entwicklung	34
6.2.1	Umsetzung des Prototyps in Python	34
6.2.2	Verbindungsaufbau und Nachrichtenempfang	34
6.2.3	Nachrichtenverarbeitung	35
6.2.4	Umgang mit großen Datenmengen	35
6.2.5	Grenzen des Prototyps	37
6.3	Konzeptplan und Umsetzung der Benutzeroberfläche	38
6.3.1	Systemarchitektur	38
6.3.2	Datenbankkonzept	39
6.3.3	Nachrichtenfluss und -Verarbeitung	40
6.3.4	Performance-Konzept	41
6.3.5	Aufbau der Benutzeroberfläche	41
7	Umsetzung	43
7.1	Realisierung der Softwarearchitektur	43

7.2	Verarbeitung eingehender MQTT-Nachrichten	43
7.3	Persistente Speicherung der Kommunikationsdaten	44
7.4	Realisierung des Paging-Konzepts	45
7.5	Teilnehmerüberwachung	47
7.6	Realisierung der Prozessanalyse	47
7.7	Benutzeroberfläche	47
8	Testkonzept und Durchführung von Tests	48
8.1	Testkonzept	48
8.1.1	Testumgebung	48
8.1.2	Funktionstests	48
8.1.3	Performance-Tests	49
8.2	Ergebnisse der Tests	50
9	Bewertung und Ergebnis	51
9.1	Bewertung der Testergebnisse	51
9.2	Umsetzung der Anforderungen	52
9.2.1	Nachrichtendarstellung und Filterung	52
9.2.2	Fehlererkennung	52
9.2.3	Teilnehmerüberwachung	52
9.2.4	Automatisierungsablauf-Erkennung	52
9.2.5	Performance	53
9.2.6	Erweiterbarkeit und Zukunftstauglichkeit	53
10	Fazit und Ausblick	54
10.1	Überblick über das Projekt	54
10.2	Zukünftige Entwicklung der Anwendung	55
10.2.1	Weitere Automatisierungsabläufe	55
10.2.2	Verbesserte Konfigurierbarkeit	55
10.2.3	Linux / HMI	55
	Quellenverzeichnis	56
	Abbildungsverzeichnis	60
	Tabellenverzeichnis	61
	A Anmerkung zur Nutzung von Künstlicher Intelligenz	62

B Ergänzungen	63
B.1 Details zu bestimmten theoretischen Grundlagen	63
B.2 Weitere Details, welche im Hauptteil den Lesefluss behindern	63
C Details zu Laboraufbauten und Messergebnissen	74
C.1 Versuchsanordnung	74
C.2 Liste der verwendeten Messgeräte	74
C.3 Übersicht der Messergebnisse	74
C.4 Schaltplan und Bild der Prototypenplatine	74
D Zusatzinformationen zu verwendeter Software	75
D.1 Struktogramm des Programmentwurfs	75
D.2 Wichtige Teile des Quellcodes	75
E Datenblätter	76
Sachwortverzeichnis	80

1 Einleitung

Für funktionierende zusammenhängende Produktionsanlagen ist der Austausch vieler Daten zwischen einzelnen Maschinen essentiell. Gerade in komplexen Verpackungslinien der Firma MULTIVAC werden ständig Daten über Zustände, Prozesse und verschiedene Ereignisse ausgetauscht, wodurch ein störungsfreier Produktionsablauf sichergestellt wird. Bei MULTIVAC wird für diese Kommunikation das Nachrichtenprotokoll Message Queuing Telemetry Transport (MQTT) eingesetzt.

Mit steigenden Anforderungen an Verpackungsmaschinen und zunehmender Anlagenkomplexität wächst auch das Nachrichtenaufkommen stark an. Dadurch wird die Analyse und Fehlerdiagnose der Kommunikationsabläufe jedoch zunehmend schwieriger. Bestehende Werkzeuge ermöglichen zwar die Anzeige einzelner Nachrichten, bieten jedoch nur begrenzte Unterstützung bei der systematischen Fehlersuche und Diagnose. Dies führt zu einem hohen Analyseaufwand und erfordert umfangreiches Expertenwissen für die speziell in MULTIVAC entwickelte MQTT-Umsetzung.

Ziel dieser Projektarbeit ist daher die Entwicklung einer Überwachungs- und Diagnose-Applikation zur Analyse der MQTT-basierten Maschinenkommunikation in Verpackungslinien. Die Anwendung soll dabei Kommunikationsabläufe transparent darstellen, Nachrichten dauerhaft speichern sowie eine effiziente Analyse großer Datenmengen ermöglichen. Dadurch soll der Analyseaufwand reduziert werden. Darüber hinaus sollen Kommunikations- und Prozessfehler automatisch erkannt sowie Funktionen zur Teilnehmerüberwachung und zur manuellen Nachrichtensimulation bereitgestellt werden. Dadurch soll der Arbeitsaufwand für die Fehlersuche bei der Inbetriebnahme verringert werden.

2 Unternehmens- und Projektumfeld

2.1 Vorstellung des Praxispartners

Die vorliegende Projektarbeit wurde im Unternehmen MULTIVAC erarbeitet. MULTIVAC hat seinen Hauptsitz in Wolfertschwenden und besitzt eine weltweite Präsenz durch zahlreiche Tochtergesellschaften und Servicestandorte. Es ist ein familiengeführtes Unternehmen und wurde 1961 gegründet. MULTIVAC ist führender Anbieter von Verarbeitungs- und Verpackungslösungen für die Bereiche Lebensmittel, Medizin- und Pharmaprodukte sowie Konsum- und Industriegüter.

Das Produkt- und Leistungsportfolio des Unternehmens umfasst Thermoformverpackungsmaschinen und Traysealer, Kennzeichnungs- und Inspektionssysteme, Automatisierungslösungen sowie komplette Verpackungs- und Prozesslinien.

In der Abteilung „Solutions“, in der die Projektarbeit realisiert wurde, werden kundenspezifische Produktions- und Verpackungslinien geplant und umgesetzt. In diesem Kontext werden verschiedene MULTIVAC-Maschinen und externe Komponenten zu einer durchgängigen Produktionslinie kombiniert. Die in dieser Arbeit erarbeitete Applikation soll hier das Testen beziehungsweise die Fehlersuche in der Maschinenkommunikation erleichtern.

2.2 Maschinenkommunikation in Produktionslinien

Die kundenspezifischen Produktionslinien sowie weitere Automatisierungslösungen setzen sich aus mehreren miteinander vernetzten Maschinen und Softwarekomponenten zusammen. Die Gewährleistung eines reibungslosen Betriebs erfordert den Austausch von Zuständen, Ereignissen und Prozessdaten zwischen den Systemen. Die Kommunikation erfolgt dabei über definierte Schnittstellen und Protokolle.

Für echtzeitkritische Steuerungsaufgaben wird typischerweise das Feldbussysteme *EtherCAT* eingesetzt. *EtherCAT* überträgt zyklisch kleine Datenmengen mit fest definierter Struktur und garantiert sowohl kurze als auch deterministische Reaktionszeiten [4, S. 287–290]. Über diesen Kommunikationsweg werden beispielsweise Sensordaten, Aktorbefehle oder Zustandsinformationen zwischen Steuerung und Maschine ausgetauscht.

Ergänzend dazu existiert eine übergeordnete Kommunikationsebene, auf welcher Informationen zwischen vollständigen Linienteilnehmern und der Liniensteuerung ausgetauscht werden. Hier kommt bei MULTIVAC das MQTT-Protokoll zum Einsatz. Im Gegensatz zu Feldbussen stehen hierbei weniger Echtzeitanforderungen im Vordergrund. Stattdessen werden größere und in ihrer Struktur variable Datenmengen übertragen. Beispiele hierfür sind Zustandsänderungen, Diagnosedaten oder Artikelinformationen.

Durch die höhere Informationsdichte sowie die Vielzahl unterschiedlicher Nachrichtentypen entsteht ein deutlich komplexerer Nachrichtenfluss. Während sich Feldbuskommunikation meist auf fest definierte Telegramme beschränkt, können MQTT-Nachrichten unterschiedlichste Inhalte besitzen und von zahlreichen Teilnehmern gleichzeitig veröffentlicht werden. Dies erschwert insbesondere bei komplexen Produktionslinien die Analyse von Kommunikationsabläufen und die systematische Fehlersuche.

2.3 Herausforderungen bei der Analyse von Kommunikationsabläufen

Bislang steht den Inbetriebnehmern und Software-Entwicklern lediglich eine Auswahl an Programmen zur Verfügung, die für die reine Aufzeichnung und Anzeige von Daten bestimmt sind. Sie wollen wenn möglich ein breites Publikum ansprechen, weswegen sie möglichst viele Domänen abdecken und somit kein Spezialwissen für den einzelnen Gebrauch zur Verfügung stellen können. Bei der Suche nach Fehlerursachen müssen daher häufig große Mengen an Nachrichten manuell durchsucht werden. Bleibt ein Fehler gänzlich unerkannt, kann das später schwerwiegende Folgen haben.

Ein weiterer Aspekt ist, dass eine Vielzahl von Inbetriebnehmern wenig Kenntnis hinsichtlich der Abläufe und des Nachrichtenaufbaus der Maschinenkommunikation verfügt. Infolgedessen sehen sich Software-Entwickler häufig veranlasst, die Maschine in der Produktionshalle aufzusuchen.

Durch den Einsatz einer Analyse- und Monitoring-Applikation kann der erforderliche Zeitaufwand erheblich reduziert werden.

3 Grundlagen

3.1 Grundprinzip MQTT

Damit verschiedene Maschinen im Linienverbund wichtige Daten austauschen können, wird eine Kommunikationsschnittstelle benötigt. Bei MULTIVAC wird dafür MQTT (Message Queuing Telemetry Transport) verwendet. MQTT ist ein leichtgewichtiges, ereignisbasiertes Nachrichtenprotokoll, welches auf dem Publish/Subscribe-Prinzip basiert [22, S. 5–6]. Das Protokoll wurde für Umgebungen mit begrenzten Ressourcen, niedriger Bandbreite und potenziell instabilen Netzwerken entwickelt und ist daher besonders für IoT- und Automatisierungsanwendungen geeignet. Das Publish/Subscribe-Modell sorgt dabei für eine räumliche, zeitliche und logische Entkopplung [22, S. 5]. Die Entkopplung wird durch die Kommunikation über einen Broker etabliert. Dadurch sind Sender und Empfänger nicht direkt miteinander verbunden, wodurch die Kommunikation nicht durch fehlende oder fehlerhafte Empfänger unterbrochen wird [12, S. 9].

3.1.1 Publisher

Jeder MQTT-Client kann sogenannter Publisher sein, der mit dem Broker verbunden ist. Er erzeugt Nachrichten und sendet diese an den Broker. Die Nachrichten werden nicht an einen konkreten Empfänger adressiert, sondern unter einem sogenannten Topic veröffentlicht. Der Publisher besitzt keine Kenntnis darüber, welche Clients die Nachricht empfangen sollen, wodurch eine logische Entkopplung zwischen Sender und Empfänger entsteht [12, S. 15–16].

3.1.2 Subscriber

Ein Client des MQTT-Brokers kann neben der Rolle als Publisher auch ein Subscriber sein. Dieser registriert sich für bestimmte Topics. Dafür sendet er eine SUBSCRIBE-Nachricht, in der jedes einzelne Topic, das der Client abonnieren will, mit dem zugehörigen Quality of Service Level angegeben wird [12, S. 16]. Der Subscriber erhält durch den Broker automatisch alle Nachrichten, die unter den von ihm abonnierten Topics veröffentlicht werden. Genau wie die Publisher keine Kenntnisse über Empfänger haben, haben die Subscriber auch keine direkten Informationen über den Sender und über die Existenz von Publishern [12]. Die Information über den Sender einer Nachricht kann dadurch erst in der Nachricht selbst freigegeben werden.

3.1.3 Broker

Der MQTT-Broker selbst ist die zentrale Instanz im MQTT-System. Er ist für den Empfang von Nachrichten seitens der Publisher zuständig, verwaltet die Topic-Abonnements und verteilt die Nachrichten an die entsprechenden Subscriber. Dabei entscheidet der Broker anhand von Topic-Filtern und Quality-of-Service-Parametern, wie und an wen eine Nachricht weitergeleitet wird [12, S. 11]. Durch den Broker kommunizieren Publisher und Subscriber niemals direkt miteinander. Da der MQTT-Broker zahlreiche Aufgaben übernimmt, benötigt dieser eine entsprechend leistungsfähige Hardware. Im Gegensatz dazu sind die Anforderungen an die Clients durch das Nutzen eines Brokers so klein, dass diese wenig performante Hardware benötigen, wodurch nahezu jedes IoT-Gerät als Client implementiert werden kann [24, S. 117–118].

Zum Nachrichtenaustausch zwischen Broker und Client nutzt MQTT grundsätzlich das TCP/IP-Protokoll. Neben TCP/IP kann MQTT auch über WebSockets betrieben werden. TCP/IP und Websocket ermöglichen dabei ein sicheres Senden von Nachrichten und erkennen Verbindungsabbrüche durch das Fehlen einer Bestätigung vom Client nach dem Senden. Damit auch der Client einen Verbindungsabbruch bemerkt, wird von ihm, sofern keine Nachrichten veröffentlicht werden, in einem festgelegten Intervall eine leere Nachricht, ein sogenanntes „PINGREQ“, gesendet. Der Broker reagiert auf diese Mitteilung mit einer sogenannten „PINGRESP“-Nachricht. Bleibt diese Nachricht des Brokers aus, erkennt der Client einen Verbindungsabbruch. Sollte die Verbindung unterbrochen sein, sendet der Broker automatisch eine „Last Will and Testament (LWT)“-Nachricht an die anderen Clients. Diese wird bereits beim Verbindungsaufbau der Clients an den Broker übermittelt und sorgt dafür, dass die anderen Clients, die das entsprechende Topic des entkoppelten Clients abonniert haben, von dem Verbindungsabbruch erfahren [24, S. 119].

3.1.4 Topics als Adressierungsmechanismus

Topics sind logische Adressen innerhalb des MQTT-Systems. Sie sind als hierarchische Zeichenkette aufgebaut, wie zum Beispiel: `machine/line1/press/status`. Dabei trennen die Schrägstriche die einzelnen Topic-Ebenen, die der Filterung der Nachrichten dienen [12, S. 19]. Für den Fall, dass mehrere Topics auf einmal oder aber alle Topics abonniert werden sollen, fungieren sogenannte Wildcards als Platzhalter. Der Zeichenfolge „#“ kommt die Funktion zu, sämtliche nachfolgenden Topic-Hierarchien zu ersetzen. Ihre Positionierung ist dabei zwingend am Ende des betreffenden Topics zu erfolgen. Demgegenüber steht die Funktion der „+“-Wildcard, die eine Auswahl aller Topics einer bestimmten Topic-Hierarchie ermöglicht [24, S. 120].

Durch diesen Aufbau wird eine strukturierte und skalierbare Organisation von Nachrichten ermöglicht und macht das System flexibel und einfach erweiterbar [12, S. 21].

3.2 Architektur der MQTT-Nachrichten

MQTT ist ein binäres Protokoll, das im OSI-Modell auf Anwendungsschichtebene ausgeführt wird. Alle MQTT-Nachrichten werden als sogenannte Control Packets übertragen, welche einem einheitlichen strukturellen Aufbau folgen. Dieser Aufbau ist bei jedem Nachrichtentyp genau gleich [21, S. 21–29].

3.2.1 Aufbau eines MQTT-Control-Packets

Eine MQTT-Nachricht besteht aus drei logisch getrennten Bereichen:

Der Fixed Header beinhaltet mindestens 2 Bytes. Das erste Byte bestimmt den Control-Packet Typ und Flags. Der Control-Packet Typ definiert die Nachricht [21, S. 30]:

- CONNECT (Verbindungsaufbau)
- PUBLISH (Nachricht senden)
- SUBSCRIBE (Topic abonnieren)

- DISCONNECT (Verbindung trennen)

Flags besitzen je nach Pakettyp unterschiedliche Bedeutungen. Beim PUBLISH-Paket enthalten die Flags unter anderem ein DUP, was eine wiederholte Übertragung bedeutet, das Quality-of-Service-Level oder ein RETAIN-Flag [12, S. 15]. Dieses RETAIN-Flag ist dafür zuständig, dass der Broker diese Nachricht speichert. Verbindet sich ein neuer Client als Subscriber und abonniert das Topic dieser Nachricht, sendet der Broker sofort automatisch die RETAIN-Nachrichten [12, S. 29]. Dadurch erhalten Clients nach der Verbindung zum Client beispielsweise die wichtigsten Informationen über den aktuellen Stand, andere Teilnehmer und vieles mehr, sobald sie sich mit dem Broker verbinden.

Das zweite bis optional fünfte Byte enthält dann, wie viele Bytes nach dem Fixed Header folgen [21, S. 30]. Die variable Kodierung zwischen zwei und fünf Bytes sorgt dafür, dass kleine Nachrichten sehr kurze Header besitzen. Dies ist ein wesentlicher Faktor für die Effizienz von MQTT, vor allem bei kleinen Payloads.

Der Variable Header enthält steuerungsrelevante Zusatzinformationen und ist abhängig vom jeweiligen Control-Packet Typ. Er enthält zum Beispiel den Topic-Namen, Protokollnamen und -Versionen und Verbindungsflags [21, S. 30–39].

Die Payload enthält die eigentlichen Nutzdaten der MQTT-Nachricht. Aufgrund der agnostischen Natur von MQTT existieren keine Vorgaben bezüglich des Datenformats oder der Struktur. Der Begriff „agnostisch“ beschreibt die Tatsache, dass MQTT über keine detaillierten Kenntnisse bezüglich des Systems verfügt, sondern lediglich die Einhaltung von Standards gewährleisten muss. Eine einheitliche Kodierung ist dabei ein mögliches Beispiel für einen Standard [42]. Mögliche Inhalte sind beispielsweise Texte, JSON-Daten oder Binärdaten. Diese sind typischerweise maximal wenige Kilobytes groß. Bei MULTIVAC ist die Payload als JSON-Dokument definiert. Die Interpretation der Payload erfolgt ausschließlich in der Anwendungsschicht, nicht durch den Broker. Dadurch ist MQTT universell einsetzbar [12, S. 15][24, S. 124].

3.2.2 Datensicherheit bei MQTT

In der Grundkonfiguration eines MQTT-Brokers ist MQTT ebenso unsicher wie HTTP. Das heißt, dass jeder, der den Port des Brokers kennt, alle Nachrichten mitverfolgen kann. Außerdem und viel schwerwiegender ist jedoch die Möglichkeit, schädliche Nachrichten unter bestimmten Topics zu

senden. Diese können beispielsweise dazu führen, dass eine gesamte Maschine kurz- oder langfristig lahmgelegt wird. Um dieses Problem zu umgehen, besteht die Möglichkeit, die Kommunikation mit SSL/TLS zu verschlüsseln [18].

Im Rahmen des TLS-Handshakes tauschen Client und Broker Informationen aus, aus denen anschließend ein gemeinsamer Sitzungsschlüssel abgeleitet wird. [14, S. 5–7].

Für die Authentifizierung von Publishern und Subscribern kann außerdem ein Benutzername und ein Passwort hinterlegt werden, damit der Broker nicht frei zugänglich ist [18].

Bei Verpackungslinien von MULTIVAC ist das Netzwerk nicht von außen zugänglich, weswegen auf Verschlüsselung und das Nutzen von Benutzername und Passwort verzichtet wird.

3.3 Quality of Service

3.3.1 Allgemeine Definition

Quality of Service (QoS) beschreibt bei MQTT die Zuverlässigkeit der Nachrichtenübertragung und betrifft somit immer die Verbindungen vom Publisher zum Broker und vom Broker zum Subscriber. Dabei können Nachrichten an beiden Stellen verloren gehen [12, S. 22]. Publisher und Subscriber können jedoch jeweils ein eigenes QoS-Level festlegen, während der Broker aber immer das QoS-Level des Subscribers verwendet, falls die QoS-Level unterschiedlich sind [12, S. 25].

3.3.2 Übersicht der Quality-of-Service-Level

MQTT definiert drei unterschiedliche QoS-Level: null, eins und zwei. Dabei steht null dafür, dass die Nachricht maximal einmal empfangen wird, eins dafür, dass die Nachricht mindestens einmal empfangen wird und zwei, dass die Nachricht genau einmal empfangen wird.

Bei Level null wird die Nachricht einmal gesendet, wodurch der Empfang nicht garantiert wird. Es gibt nämlich keine Empfangsbestätigung, ein sogenanntes Acknowledgement. Da aber nur einmal eine Nachricht gesendet wird, ist dieses Level sehr effizient. Dadurch eignet sich dieses QoS-Level

insbesondere für hochfrequente Sensor- oder Telemetriedaten, bei denen der Verlust einzelner Datenpakete unkritisch ist. [12, S. 22].

Bei Level eins wird die Nachricht mindestens einmal zugestellt. Der Publisher sendet die Nachricht so lange erneut, bis ein Acknowledgement namens PUBACK empfangen wird. Währenddessen wird die Nachricht beim Publisher zwischengespeichert. Sollte der Publisher nach dem ersten Senden kein PUBACK empfangen, sind doppelte Zustellungen möglich. Das kann zum Beispiel durch Verbindungsabbrüche geschehen. Kommt eine Empfangsbestätigung wegen eines Verbindungsabbruchs nicht an, sendet der Publisher dieselbe Nachricht erneut, obwohl sie beim Broker schon angekommen ist. Damit ist das QoS-Level 1 ein guter Kompromiss zwischen Zuverlässigkeit und Performance. Typische Anwendungen für dieses Level sind Alarmer, Statusmeldungen und sicherheitsrelevante, aber Duplikat-tolerante Daten [12, S. 23].

Bei Level zwei werden Nachrichten genau einmal zugestellt. Diese Zusicherung wird durch einen vierstufigen Handshake sichergestellt. Dabei empfängt der Publisher nach dem Senden ein Acknowledgement für das Senden und sendet daraufhin ein weiteres Acknowledgement zurück. Die vierte Stufe ist dann ein weiteres Acknowledgement PUBCOMP zur Sicherstellung, dass wirklich alle Acknowledgements angekommen sind. Damit besitzt Level 2 die höchste Zuverlässigkeit, ist aber durch die hohe Netz- und Brokerlast auch ineffizient. Somit eignet sich dieses QoS-Level für kritische Nachrichten, die sicher, aber niemals doppelt ankommen dürfen [12, S. 23–24].

Die Auswahl des passenden QoS-Levels hängt dadurch von der Kritikalität der Nachricht, Sendefrequenz und der Toleranz gegenüber Datenverlust oder Duplikaten ab. Neben den reinen Nachrichteneigenschaften spielt die gesamte System- und Brokerlast bei der Auswahl des Levels auch eine entscheidende Rolle. Dabei ist ein höheres QoS-Level nicht immer besser [12, S. 25].

3.4 Vorteile von MQTT in der Automatisierung

MQTT ist durch den sehr kleinen Nachrichten-Header, der aus nur zwei bis fünf Bytes besteht, ein sehr leichtgewichtiges Protokoll. Dadurch benötigt es nur geringe Bandbreite, wodurch es sich vor allem für ressourcenarme Geräte und IoT-Sensoren eignet [9, S. 68–69].

Durch das Publish-/Subscribe-Modell entsteht eine Entkopplung von Sendern und Empfängern, wodurch keine Geräte-zu-Geräte-Kommunikation notwendig ist. So wird der Netzwerkverkehr im Vergleich zu Polling-Methoden wie HTTP oder OPC UA (Open Platform Communications Uni-

fied Architecture) sehr stark reduziert [28, S. 31–32]. Außerdem kann der Broker eine große Anzahl an Clients verwalten und unterstützt bis zu tausend Verbindungsteilnehmer gleichzeitig. Dadurch ist die Erweiterung von Systemen ohne Eingriffe an den bestehenden Komponenten möglich [12, S. 11]. Aufgrund seiner Payload-Agnostik erlaubt MQTT eine anwendungsagnostische Datenübertragung und kann somit flexibel an unterschiedliche Einsatzszenarien angepasst werden. Das erleichtert auch die mögliche Weiterentwicklung der Anwendungsschicht, da diese klar von der Kommunikationsschicht getrennt ist [13].

Durch die flexible Anpassung der Daten-Kritikalität durch die Quality-of-Service-Stufen ist MQTT sehr gut für instabile Netzwerke geeignet. Dazu hilft auch die Unterstützung für die Session-Wiederaufnahme, genauso wie die Zwischenspeicherung der Nachrichten und der Last-Will-Mechanismus zur Statusüberwachung [9, S. 73–74].

3.5 .Net als Laufzeitplattform

Im Rahmen dieser Arbeit wird *.NET* als Ausführungsplattform eingesetzt. Das „.Net-Framework ist eine Technologie, die die Erstellung und Ausführung von Windows-Apps und Webdiensten unterstützt“ [36]. Es besteht aus der Common Language Runtime (CLR) und der .NET-Framework-Klassenbibliothek. Die CLR bildet die Laufzeitumgebung von *.NET*. Sie ist für die Programmausführung, die Speicherverwaltung, das Thread-Management sowie sicherheitsrelevante Funktionen verantwortlich. Die Klassenbibliothek stellt eine Vielzahl objektorientierter Typen und Funktionen bereit, die bei der Entwicklung von Anwendungen genutzt werden können. Dazu zählen unter anderem Bibliotheken für Netzwerkkommunikation, Datenbankzugriffe und die Entwicklung grafischer Benutzeroberflächen. Darüber hinaus umfasst .NET verschiedene Frameworks zur Erstellung von Benutzeroberflächen, beispielsweise Windows Presentation Foundation (WPF) und .NET MAUI [36].

3.6 XAML und Deklarative Benutzeroberflächen

XAML (eXtensible Application Markup Language) ist eine deklarative, XML-basierte Beschreibungssprache zur Definition grafischer Benutzeroberflächen in .NET-basierten Graphical User Interface (GUI)-Frameworks wie WPF und WinUI. Sie dient dazu, die Struktur und das visuelle

Erscheinungsbild einer Oberfläche unabhängig von der zugrunde liegenden Anwendungslogik zu beschreiben [39, S. 17–18].

Im Gegensatz zur imperativen UI-Erstellung im Quellcode werden in XAML Benutzeroberflächen in hierarchische Strukturen von UI-Elementen formuliert. Dies ermöglicht eine klare Entkopplung zwischen Darstellung und Programmlogik, da die Interaktions- und Verarbeitungslogik in Programmiersprachen wie C# implementiert wird, während XAML ausschließlich die Oberfläche beschreibt [39, S. 17–18].

Die in XAML beschriebenen Elemente werden zur Laufzeit durch das jeweilige Framework instanziiert und mit den zugehörigen Programmkomponenten verknüpft. In Verbindung mit Mechanismen wie Data Binding unterstützt XAML die Umsetzung strukturierter Softwarearchitekturen und fördert eine lose Kopplung zwischen Benutzeroberfläche und Anwendungslogik [1] [15].

Durch diese Eigenschaften verbessert XAML die Wartbarkeit, Erweiterbarkeit und Wiederverwendbarkeit grafischer Anwendungen, da Änderungen an der Benutzeroberfläche weitgehend unabhängig von der zugrunde liegenden Programmlogik vorgenommen werden können.

3.7 Model-View-ViewModel als Architekturstruktur

MVVM (Model-View-ViewModel) ist ein Architekturmuster zur strukturierten Entwicklung von Anwendungen mit grafischer Benutzeroberfläche. Ziel des Musters ist die Trennung von Darstellung, Anwendungslogik und Datenmodell, wodurch die Wartbarkeit, Erweiterbarkeit und Testbarkeit einer Anwendung verbessert werden [30].

Insbesondere bei datengetriebenen Anwendungen mit umfangreicher Benutzerinteraktion steigt die Komplexität der Softwarearchitektur. Ohne eine klare Trennung der Verantwortlichkeiten kann dies zu einer starken Kopplung zwischen Benutzeroberfläche und Anwendungslogik führen. Architekturmuster wie MVVM begegnen diesem Problem durch die Aufteilung der Anwendung in klar definierte Komponenten [30].

Das MVVM-Muster unterteilt eine Anwendung in die drei Hauptbestandteile Model, View und View-Model. Das Model repräsentiert die fachlichen Daten. Die View definiert die grafische Benutzeroberfläche und dient ausschließlich der Darstellung der Daten. Das View-Model fungiert als

Vermittlungsschicht zwischen Model und View. Es stellt der Benutzeroberfläche die benötigten Daten und Befehle bereit und verarbeitet Benutzerinteraktionen.

Die Kommunikation zwischen View und View-Model erfolgt üblicherweise über Data Bindings. Änderungen an den Datenmodellen können dadurch automatisch in der Benutzeroberfläche dargestellt werden, ohne dass eine direkte Kopplung zwischen den Komponenten erforderlich ist. Dies reduziert den Implementierungsaufwand und erleichtert spätere Erweiterungen der Anwendung [17].

Durch die konsequente Trennung von Zuständigkeiten unterstützt MVVM die Entwicklung modularer Anwendungen und stellt insbesondere bei Anwendungen mit umfangreicher Benutzeroberfläche eine etablierte Architekturstruktur dar [30].

4 Ausgangssituation

4.1 Monitoring über MQTT.fx

Die grafische und leicht verständliche, aber gleichzeitig leistungsfähige Anwendung MQTT.fx wird bisher für das Testen und Debuggen von MQTT-basierter Kommunikation verwendet. Sie wird verwendet, um die Nachrichtenflüsse zwischen Clients und Broker sichtbar zu machen [19]. Diese Software ist eine von zwei Möglichkeiten bei MULTIVAC, den MQTT-Nachrichtenverlauf darzustellen und auch Nachrichten zu simulieren.

4.1.1 Funktionalität von MQTT.fx

MQTT.fx ist ein grafischer Desktop-Client für das MQTT-Protokoll. Die Anwendung verbindet sich als MQTT-Client mit dem externen Broker. Beim Verbinden erlaubt die Applikation das Anlegen wiederverwendbarer Verbindungsprofile. Konfigurierbare Parameter sind dabei die Broker-Adresse, Port, Client-ID, MQTT-Version, Clean-Session, Keep-Alive-Zeit, Authentifizierung durch Benutzername und Passwort und eine TLS/SSL-Verschlüsselung.

Über einen Reiter Publish lassen sich Nachrichten über frei definierbare Topics versenden. Dazu lassen sich auch das QoS-Level und die Retain-Flag manuell einstellen. Nachrichten können ähnlich wie Verbindungsprofile über Vorlagen oder die Historie wiederverwendet werden. So können Sensorwerte, Statusmeldungen und Steuerbefehle simuliert werden [8].

Neben dem Reiter Publish gibt es auch einen Subscribe-Reiter (4.1). Hier können einzelne Topics oder Topic-Hierarchien abonniert werden. Die Darstellung der empfangenen Nachrichten unter den abonnierten Topics erfolgt dann in Echtzeit und mit den relevanten Metadaten Topic, Payload, QoS und Empfangszeitpunkt.

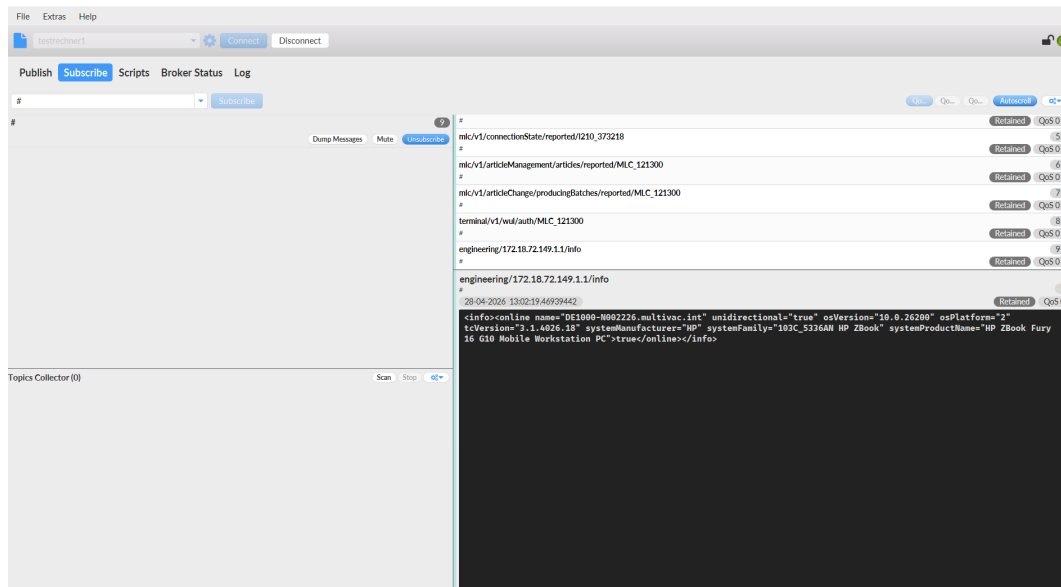


Abbildung 4.1: Subscribe-Reiter in MQTT.fx

Darüber hinaus verfügt das Programm über eine Logging- und Analysefunktion. Dieser Reiter ist mit einem integrierten Log-Fenster ausgestattet, welches den Verbindungsaufbau und -abbau, Subscribe- und Publish-Events, MQTT-Nachrichten sowie Broker-Statusmeldungen protokolliert.

Eine weitere wichtige Funktion stellt die Skript- und Automatisierungsfunktion dar. MQTT.fx stellt eine einfache Java-basierte Skript-Schnittstelle bereit. Diese ermöglicht automatisiertes Publizieren bestimmter Nachrichtenverläufe, die manuell durch einen Java-Programmcode erstellt werden können [10].

Zuletzt wird neben den zuvor genannten Reitern der Broker-Status angezeigt. In diesem werden grundsätzliche Informationen über die Anzahl der verbundenen Clients, Anzahl an gesendeten Nachrichten und weitere ähnliche Daten des Brokers angezeigt.

MQTT.fx stellt eine umfangreiche Sammlung von Funktionen zur Verfügung, die insbesondere für das Testen, Debuggen und Analysieren von MQTT-Kommunikation geeignet sind. Der Funktionsumfang ist dabei bewusst auf die Rolle eines Client-Werkzeugs ausgelegt, wobei der produktive Einsatz als dauerhaftes Monitoring- oder Visualisierungssystem nicht intendiert ist.

4.1.2 Gründe für und gegen den Einsatz von MQTT.fx

Diese Applikation ermöglicht es Entwicklern, ohne eigene Implementierung Publisher- und Subscriber-Rollen zu simulieren. Ein klarer Vorteil bei MQTT.fx ist die mögliche Darstellung von Clean Session, Last-Will-Nachrichten, QoS-Level oder Retained Messages. Darüber hinaus bietet MQTT.fx umfangreiche Logging- und Visualisierungsfunktionen, wodurch falsche Zustandsannahmen oder Timing-Probleme schnell erkannt werden können. MQTT.fx bietet zudem unterschiedliche Payload-Decoder, wodurch eine bessere Analyse verschiedener MQTT-Nachrichten, ohne eigene Interpretation dieser, möglich ist [10].

MQTT.fx speichert die empfangenen Nachrichten nicht dauerhaft. Eine langfristige Analyse ist dadurch nur durch manuelles Kopieren der Logs in externe Applikationen möglich. Außerdem ist MQTT.fx nur auf die Anzeige einzelner Subscriptions fokussiert. Deshalb ist die Darstellung zusammenhängender Nachrichtenflüsse über mehrere Topics hinweg sehr eingeschränkt und erschwert die Analyse komplexer Topic-Hierarchien, wie sie in Automatisierungssystemen häufig auftreten. Ein weiterer zentraler Punkt, der gegen die Nutzung von MQTT.fx spricht, ist die Konzipierung als eigenständige Desktop-Anwendung. Deswegen ist eine Integration in bestehende HMIs oder produktive Visualisierungssysteme nicht möglich. Außerdem gibt es keine Möglichkeit zur Weiterverarbeitung von Daten, systematischen Auswertung von Tests oder zur Integration externer Datenquellen. Dadurch werden sowohl die Analyse als auch die Simulation bestimmter Szenarien eingeschränkt.

4.2 Monitoring über MQTT Explorer

Eine weitere Möglichkeit für das Auslesen von MQTT-Nachrichten bietet „MQTT Explorer“. Dieser bietet gegenüber MQTT.fx zusätzliche Funktionen und ist insgesamt übersichtlicher sowie stärker auf den Anwendungsfall ausgerichtet.

Wie in der Abbildung 4.2 zu sehen ist, listet dieser alle erkannten Topics automatisch in einer Baumstruktur auf. Dadurch ist ein ausgiebiges Wissen über die Topicstruktur oder ein vorheriges Scannen von Topics wie bei MQTT.fx nicht erforderlich, damit man die Nachrichten topic-bezogen filtern kann. Auf der rechten Seite befindet sich dann zum einen ein Nachrichtenfenster, in dem die angeklickte Nachricht in der Baumstruktur angezeigt wird. Zum anderen kann man dort auch Nachrichten selbst unter einem selbst eingegebenen Topic publizieren.

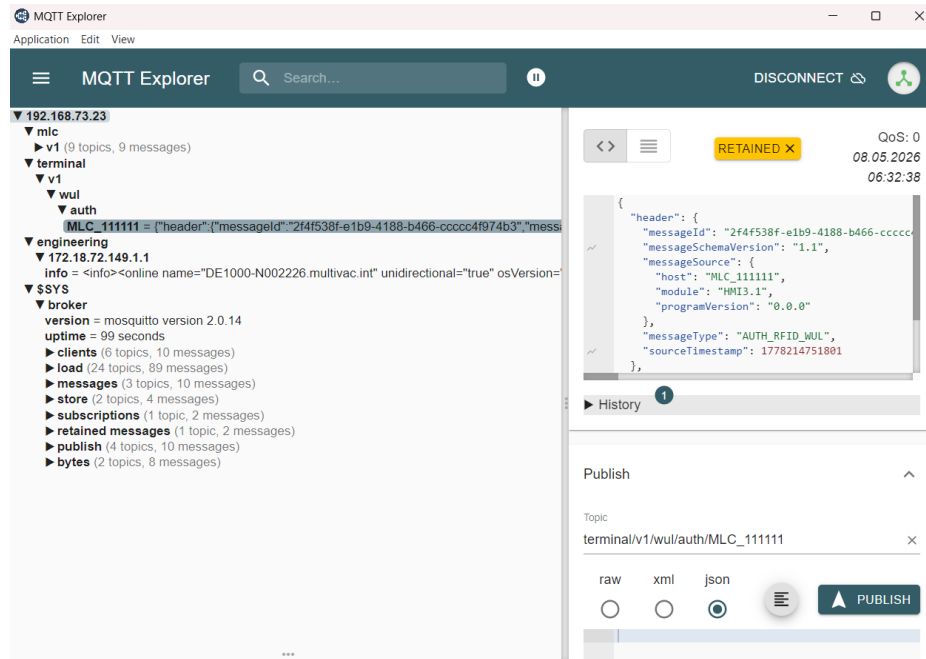


Abbildung 4.2: MQTT Explorer

Ein wesentlicher Vorteil gegenüber MQTT.fx ist die Möglichkeit der Volltextsuche, die in der Abbildung 4.2 oben als Sucheingabe-Feld zu sehen ist. Durch das Eingeben eines Begriffs werden nur noch die Nachrichten in der Baumstruktur angezeigt, die den Suchbegriff enthalten. So wird die Suche erheblich vereinfacht.

Ein großer Nachteil ist jedoch die eingeschränkte Übersichtlichkeit des Nachrichtenverlaufs. Er lässt sich zwar durch die sogenannte Historie anzeigen. Diese beschränkt sich dann aber nur auf ein gezieltes Topic, wodurch die Reihenfolge aller Nachrichten nicht vollständig nachvollzogen werden kann. Auch die Volltextsuche beschränkt sich nur auf aktuelle Nachrichten und kann vergangene Nachrichten nicht weiter nachvollziehen.

Dadurch wird die Analyse von Datenströmen, bei denen die Reihenfolge eingehender Nachrichten eine entscheidende Rolle spielt, deutlich erschwert. Aufgrund dessen erfüllt auch diese Applikation die definierten Anforderungen nicht.

4.3 Möglichkeit der Nutzung und Erweiterung bereits existenter Applikationen

Eine weitere Möglichkeit neben der Verwendung von bereits fertigen Programmen ist die Weiterentwicklung existenter Applikationen, die den Programmcode veröffentlichen. Eine hierfür besonders geeignete Applikation ist MQTTMultimeter [7].

4.3.1 Funktionsweise von MQTTMultimeter

Als eine schon sehr ausgereifte Applikation erweist sich „MQTTMultimeter“ [7]. Dieses Programm bietet eine geeignete Kombination aus MQTT.fx und dem MQTT Explorer. Es hat einen Reiter für die Anzeige der Baumstruktur, die sehr ähnlich funktioniert wie bei dem MQTT Explorer (siehe Abbildung 4.3). Auch hier lässt sich der Nachrichtenverlauf nur durch eine Historie eines einzelnen Topics darstellen.

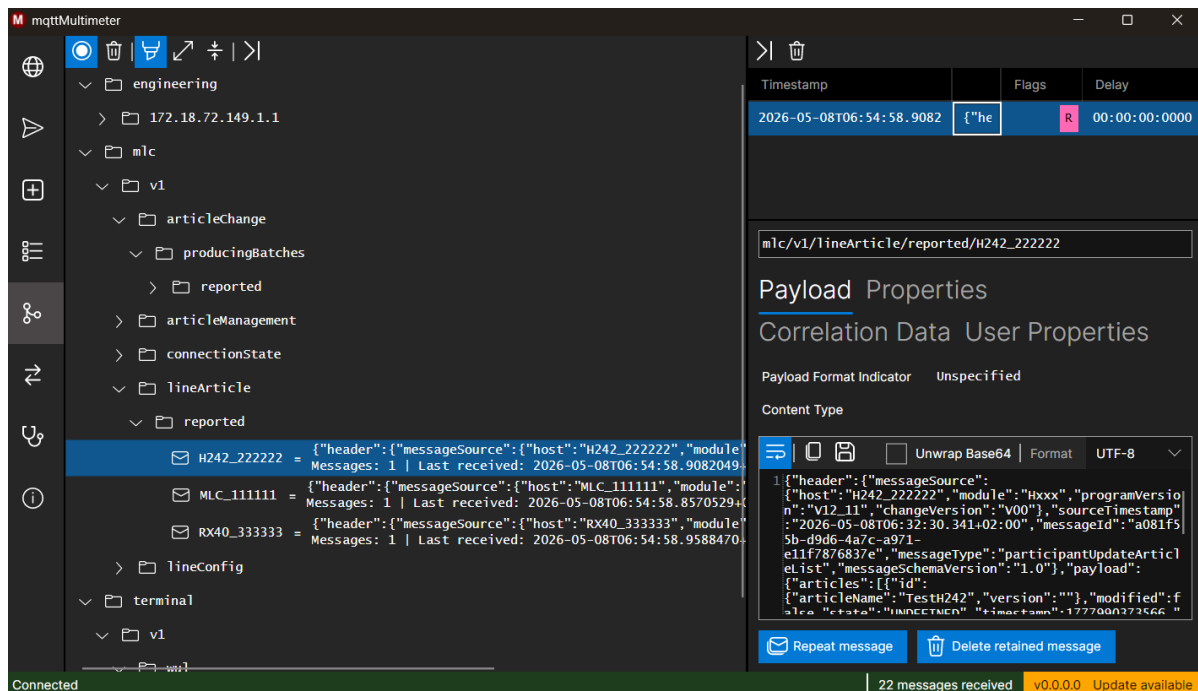


Abbildung 4.3: Anzeigen der Nachrichten in einer Baumstruktur im MQTTMultimeter

In einem anderen Reiter können dann die Nachrichten in ihrer zeitlichen Empfangsreihenfolge angezeigt werden. Der Vorteil hier gegenüber MQTT.fx ist das Filtern nach Topics durch eine Eingabe oben (siehe Abbildung: 4.4). Durch einen Klick auf das ausgewählte Topic wird die

Nachricht sowie auch weitere Informationen zur Nachricht rechts angezeigt. Hier können die Nachrichten auch in einen Ordner abgelegt werden, jedoch auch nur wieder von der Applikation selbst aufgerufen werden.

In der Baumstruktur wie auch in beim Nachrichtenverlauf kann die angezeigte Nachricht direkt in den Publish Reiter kopiert werden. Dort kann die Nachricht selbst versendet werden, wodurch das Simulieren von Nachrichten vereinfacht wird.

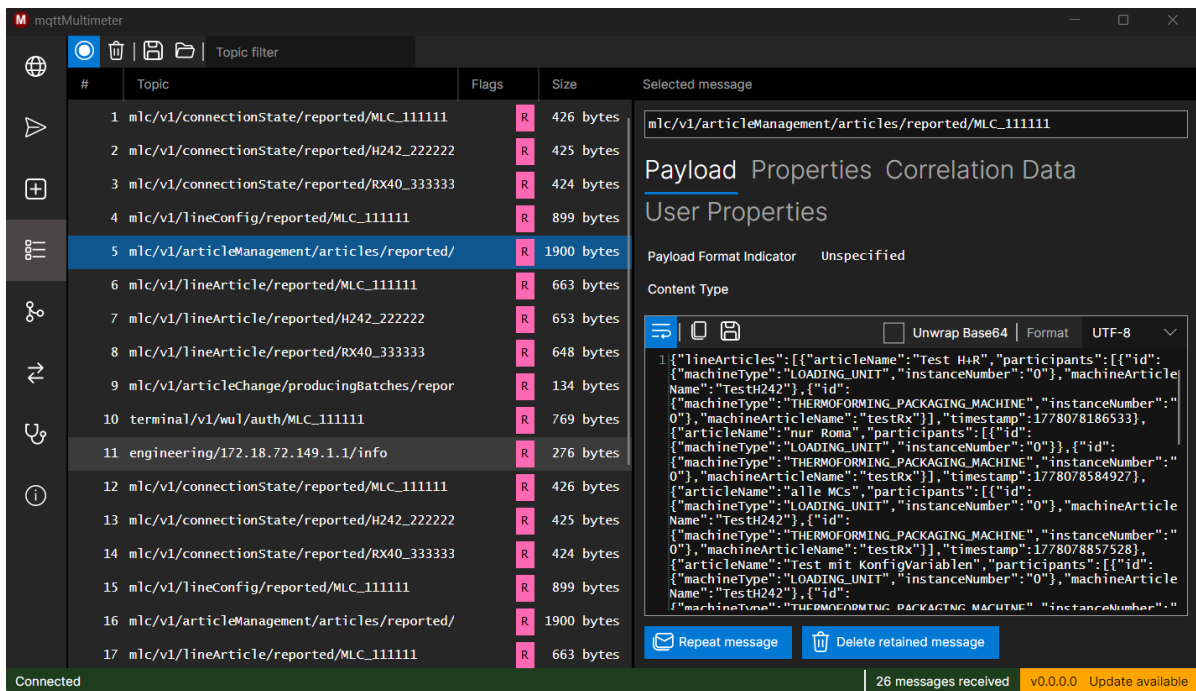


Abbildung 4.4: Nachrichtenverlauf im MQTTMultimeter

Ein Nachteil dieser Applikation ist die teilweise komplexe Benutzeroberfläche. Während beim MQTT Explorer die Nachrichten automatisch nach dem Verbinden angezeigt werden, muss hier dafür erst ein Topic abonniert werden. Des Weiteren muss in den beiden Empfangsreitern ein Haken gesetzt werden, damit die Applikation anfängt, die Nachrichten aufzuzeichnen. Ein weiterer Nachteil ist das Speichern in App-eigenen Dateien, die nicht in einem Editor geöffnet werden können, sondern nur in MQTTMultimeter selbst importiert werden können. Außerdem löscht die Applikation ab der tausendsten Nachricht die ersten empfangenen Nachrichten. Dies kann zwar durch das Setzen einer Variable auf eine andere Zahl geändert werden, ändert aber nichts an dem Verlust von Daten nach einer gewissen Zeit. Trotzdem erfüllt MQTTMultimeter die Anforderungen an ein Monitoring-Werkzeug am umfassendsten.

4.3.2 Technische Realisierung von MQTTMultimeter

MQTTMultimeter ist in C# mit dem .NET-Framework programmiert. Es ist modular aufgebaut und in mehreren Klassen aufgeteilt. Im Folgenden werden die zentralen Komponenten exemplarisch erläutert.

Die MQTT-Kommunikation wird in der Klasse `MqttClientService` gekapselt (siehe Listing: 4.1), welche als zentrale Schnittstelle zwischen Anwendung und Broker dient. Der Verbindungsaufbau erfolgt über die Bibliothek *MQTTnet* unter der Verwendung eines Builder-Patterns zur Konfiguration der Client-Optionen. Dadurch wird eine flexible und erweiterbare Initialisierung ermöglicht. Ein zentrales Merkmal der Implementierung ist die Ereignis-basierte Verarbeitung eingehender Nachrichten. Der MQTT-Client registriert hierfür Callback-Handler, die beim Empfang einer Nachricht automatisch ausgeführt werden. Dies entspricht dem asynchronen Kommunikationsmodell MQTT. Zur Gewährleistung der Thread-Sicherheit bei gleichzeitigem Empfang mehrerer Nachrichten wird ein atomarer Zähler `Interlocked.Increment(ref \_receivedMessagesCount)` verwendet. Das heißt, dass die Erhöhung des Zählers atomar ausgeführt wird, wodurch konkurrierende Zugriffe mehrerer Threads nicht zu inkonsistenten Zählerständen führen können. Die Weiterverarbeitung der Nachricht erfolgt über ein internes Event-System, wodurch eine lose Kopplung zwischen Kommunikationsschicht und Benutzeroberfläche erreicht wird.

Listing 4.1: MQTT-Kommunikation in der Klasse `MqttClientService` im `MQTTMultimeter`

```
1 public async Task<MqttClientConnectResult> Connect(ConnectionItemViewModel item)
2 {
3     _mqttClient = new MqttClientFactory().CreateMqttClient();
4     var options = new MqttClientOptionsBuilder()
5         .WithClientId(item.SessionOptions.ClientId)
6         .WithTcpServer(item.ServerOptions.Host,
7             item.ServerOptions.Port)
8         .WithCredentials(item.SessionOptions.UserName,
9             item.SessionOptions.Password)
10        .Build();
11
12    _mqttClient.ApplicationMessageReceivedAsync += OnApplicationMessageReceived;
13    _mqttClient.DisconnectedAsync += OnDisconnected;
14
15    return await _mqttClient.ConnectAsync(options);
16 }
17 async Task OnApplicationMessageReceived(MqttApplicationMessageReceivedEventArgs
18     eventArgs)
```

```
19 Interlocked.Increment(ref __receivedMessagesCount);  
20 await __applicationMessageReceivedEvent.InvokeAsync(eventArgs);  
21 }  
22
```

Die Aufgabe der Klasse *InflightPageViewModel* besteht in der Verarbeitung und Darstellung eingehender Nachrichten in der richtigen Empfangsreihenfolge (siehe Listing: B.1). Sie stellt eine Verbindung zwischen der Kommunikationsschicht (*MqttClientService*) und der UI her und übernimmt damit eine zentrale Rolle im Datenfluss der Anwendung. Eingehende Nachrichten werden ereignisbasiert verarbeitet. Dazu registriert sich das View-Model auf das Ereignis *ApplicationMessageRecieved* des MQTT-Services. Sobald eine neue Nachricht empfangen wird, wird diese asynchron entgegengenommen und in eine interne Datenquelle überführt (*SourceList <InflightPageItemViewModel> _itemsSource*). Diese Datenquelle enthält alle aktuell erfassten Nachrichten. Ein wesentliches Merkmal der Implementierung ist die Verwendung reaktiver Programmierung. Änderungen an der Benutzereingabe, insbesondere am Filtertext, werden nicht unmittelbar verarbeitet, sondern als kontinuierlicher Datenstrom behandelt. Durch den Einsatz eines Throttle-Operators wird die Verarbeitung verzögert, wodurch eine übermäßige Belastung des Systems verhindert wird. Die eigentliche Filterung der Nachrichten nach Topics oder nach einem eingegebenen Suchbegriff erfolgt innerhalb der Datenpipeline. Dabei wird ein dynamisch erzeugter Filter auf die Datenquelle angewendet, während die zugrundeliegende Liste dabei unverändert bleibt. Die Verarbeitung erfolgt vollständig asynchron, wobei Aktualisierungen der Benutzeroberfläche explizit im UI-Thread durchgeführt werden. Dadurch wird auch bei einer hohen Anzahl eingehender Nachrichten stabil und performant dargestellt.

Die programmiertechnische Umsetzung basiert auf einer klar strukturierten Architektur mit einer Trennung von Kommunikation, Datenverarbeitung und Darstellung. Durch den Einsatz asynchroner und reaktiver Konzepte wird eine performante Verarbeitung der MQTT-Nachrichten sowie eine dynamische Aktualisierung der Benutzeroberfläche erreicht. Insgesamt ermöglicht dies eine effiziente, skalierbare und wartbare Realisierung der Anwendung.

4.3.3 Gründe gegen die Erweiterung von MQTTMultimeter

Während diese Applikation durch das Speichern von Nachrichten und Nachrichtenvorlagen sowie die Darstellung der Nachrichten sowohl in zeitlicher Reihenfolge als auch in Baumstruktur viele der Anforderungen an das Monitoring-Tool erfüllt, erweist sich der Programmcode als nur schwer erweiterbar. Bei sehr hohen Nachrichtenfrequenzen gelingt es ihr nicht, alle Nachrichten anzuzeigen

und trennt sogar die Verbindung zum Broker. Dadurch wären grundlegende Änderungen in den Service-Komponenten erforderlich. Dieses Vorgehen erweist sich als sehr aufwendig [32], da der Programmcode eng mit den bestehenden Funktionalitäten verbunden ist. Das erhöht zum einen das Risiko von Fehlern und zum anderen die Komplexität, den Programmcode zu verstehen.

Deshalb wird in dieser Projektarbeit ein neues Programm von Grund auf neu erstellt. Die Architektur und Funktionsweise von MQTTMultimeter fließen jedoch in die Konzeption der zu entwickelnden Anwendung ein.

5 Technische Rahmenbedingungen

5.1 Anforderungen an die Programmiersprache

Bei der Auswahl einer geeigneten Programmiersprache sind zahlreiche Eigenschaften und Anforderungen zu berücksichtigen. Die Objektorientierung in der Programmierung ist essenziell für die Trennung der Netzwerk-, Logik- und Darstellungsschicht. Für die Anwendung ergibt sich, dass ein Zugriff auf asynchrone Programmiermodelle erforderlich ist, da MQTT ereignis- und nachrichtenbasiert arbeitet. Zur Erleichterung der Programmierung müssen für die genutzte Sprache MQTT-Client-Bibliotheken zur Verfügung stehen. Mit diesen können Prozesse wie Subscribe, Publish, Connect oder Nachrichtenempfang mit nur wenigen Zeilen Programmcode programmiert werden. Neben einer MQTT-Bibliothek sollte die Programmiersprache auch eine einfache Entwicklung grafischer Benutzeroberflächen ermöglichen und plattformübergreifend unterstützt werden.

5.1.1 Eignung der Sprache C# für die Umsetzung

Für die Umsetzung der Applikation eignet sich insbesondere die Programmiersprache C#. Sie bietet die Möglichkeit, Ereignisse asynchron zu verarbeiten, was bei großen Datenmengen eine entscheidende Rolle spielt. Dadurch werden andere Vorgänge beziehungsweise Prozesse nicht blockiert und die GUI bleibt bedienbar. Diese nicht blockierende Verarbeitung wird über die Befehle `await` beim Aufruf von Funktionen und `async` bei der Funktionsdeklaration möglich [27, S. 334–343].

Neben der asynchronen Verarbeitung bietet C# außerdem ein großes Angebot an Bibliotheken, wodurch Datenbankzugriffe und Netzwerkverbindungen programmtechnisch wesentlich vereinfacht werden.

C# wird häufig für die Entwicklung technischer Desktop-Anwendungen verwendet, da es im Gegensatz zu vielen anderen Programmiersprachen wie Java eine tiefere Einbindung in das .NET-Ökosystem bietet. Verschiedene Frameworks wie Windows Presentation Foundation oder .NET MAUI erleichtern die Entwicklung moderner Anwendungen erheblich. Insbesondere WPF wurde für die Entwicklung von Windows-Anwendungen konzipiert. [35, S. 5–6].

Deshalb wird für die Erstellung der Monitoring- und Diagnose-Software die Programmiersprache C# verwendet.

5.1.2 MQTT-Client-Bibliothek

Die eingesetzte MQTT-Bibliothek muss eine Unterstützung für MQTT 5.0 anbieten und dabei eine vollständige Implementierung der MQTT-Kernfunktionen bereitstellen. Die Kernfunktionen bestehen dabei vor allem aus *Connect* und *Disconnect*, *Subscribe* und *Unsubscribe* und dem Veröffentlichenden von Nachrichten. Außerdem sollte sie ähnlich wie bei MQTT.fx Quality-of-Service-Level, Retained Messages, Last-Will-Nachrichten und Clean Sessions abbilden können.

Diese Funktionen können durch die Bibliothek *MQTTnet* durch wenige Zeilen Programmcode vereinfacht in C# umgesetzt werden [31]. Durch diese Bibliothek lassen sich auch Ereignis- und Callback-Mechanismen über Event-Handler implementieren. Callbacks sind registrierte Rückruf-funktionen, die vom System oder Framework automatisch aufgerufen werden, sobald ein Ereignis eintritt. Dadurch können nach dem Abonnieren von Topics empfangene Nachrichten ereignisbasiert verarbeitet werden, ohne dass eine direkte Kopplung zwischen Netzwerk- und Anwendungslogik erforderlich ist. Somit ist eine Trennung zwischen Netzwerk- und Grafik-Logik möglich. Dies ist notwendig, um hohe Nachrichtenraten effizient verarbeiten zu können, ohne ständig die Benutzeroberfläche zu blockieren [40, S. 113]. Neben dem Empfangen der Nachrichten deckt die Bibliothek auch das Senden von Nachrichten unter beliebigen Topics ab [31].

5.1.3 Datenbank-Bibliothek

„Eine Datenbank ist eine Kollektion von Daten, die in einer Datenbasis gespeichert sind“ [29, S. 50]. Neben der reinen Speicherung verwalten Datenbanken auch gleichzeitig Benutzeranfragen, die von dem Datenbankmanagementsystem bearbeitet werden [29, S. 50]. Für diese Projektarbeit stehen viele verschiedene Datenbankmanagementsysteme zur Auswahl, darunter SQLite, MySQL

und Microsoft SQL Server. Diese werden im Folgenden vorgestellt und hinsichtlich ihrer Vor- und Nachteile bewertet:

SQLite ist eine leichtgewichtige eingebettete Datenbank, die direkt durch eine Bibliothek in die Anwendung integriert werden kann, ohne dafür einen Server zu implementieren [16, S. 1–2]. Durch die Speicherung von Daten genügt ein einziges Dokument, wodurch die Portabilität der Anwendung nicht durch die Datenbank erschwert wird und Daten einfach gesichert und übertragen werden können [16, S. 3–4].

MySQL ist ein serverbasiertes Datenbankmanagementsystem, das aus den drei Ebenen „Client“, „Server“ und „Speicher“ besteht [37, S. 4]. Neben der Implementierung eines Servers sind einige weitere Implementierungsschritte zur funktionstauglichen Nutzung der Datenbank nötig [37, S. 5–9]. Während zwar MySQL durch diese verschiedenen Implementierungsschritte viele weitere Funktionen wie Sicherheit durch Reihenisolation oder die Benutzung der Datenbank von mehreren Benutzern gleichzeitig mit sich bringt, ist es mit einem höheren Ressourcenbedarf verbunden. [37, S. 41–42].

Microsoft SQL Server benötigt ähnlich wie MySQL auch die Implementierung einer oder mehrerer Serverinstanzen [23, S. 36–37]. Hier ist der Aufwand einer allgemeinen Implementierung im Gegensatz zu SQLite deutlich höher, wie man schon an den Systemvoraussetzungen [23, S. 30], aber auch an den verschiedenen zu installierenden Werkzeugen sehen kann [23, S. 31–56]. Neben der schon aufwendigen Implementierung des Servers kommt die Implementierung in C# dazu. Dafür muss erst eine Verbindung zum Server aufgebaut werden [11]. Im Gegensatz dazu genügt bei SQLite ein Befehl zur Übergabe des Pfades der Datenbankdatei.

Die aufwendige Serverimplementierung bei MySQL und SQL Server führt dann aber auch dazu, dass mehrere Schreibzugriffe gleichzeitig möglich sind. Daneben werden Schreibzugriffe bei SQLite serialisiert, wodurch die Performance bei mehreren Zugriffen zur selben Zeit sinkt.

Da die entwickelte Monitoring-Applikation ausschließlich lokal auf einem einzelnen Rechner betrieben wird und kein Mehrbenutzerbetrieb vorgesehen ist, bieten die zusätzlichen Funktionen von MySQL beziehungsweise Microsoft SQL Server keinen wesentlichen Mehrwert. Aufgrund dessen kann über die Nachteile der Serialisierung von Schreiboperationen und die daraus entstehenden Performance-Einbußen hinweggesehen werden. Wegen der serverlosen Architektur, des geringen Installationsaufwands sowie der einfachen Bereitstellung wird daher SQLite als Datenbankmanagementsystem ausgewählt. Da sowohl SQLite als auch die anderen vorgestellten Datenbankmanagementsysteme Datenbankzugriffe in SQL modelliert, ist eine Erweiterung mit anderen Manage-

mentssystemen in Zukunft möglich.

Zur Integration in C# wird die Bibliothek Microsoft.Data.Sqlite verwendet. Diese stellt eine moderne und leichtgewichtige Schnittstelle zur SQLite-Datenbank bereit und unterstützt asynchrone Datenbankzugriffe, wodurch eine effiziente und nicht-blockierende Verarbeitung der Daten gewährleistet werden kann. Darüber hinaus integriert sich die Bibliothek nahtlos in das .NET-Ökosystem [2].

5.1.4 Graphical User Interface Framework

Für die Umsetzung einer Monitoring- und Visualisierungsapplikation muss das GUI-Framework ein ereignisgesteuertes Modell mit einer zentralen Ereignisschleife (Eventloop) bereitstellen. Dazu muss es den User-Interface-(UI)-Thread durch Callbacks von den Hintergrundoperationen wie Netzwerkkommunikation und Datenverarbeitung trennen, aber auch die Möglichkeit bieten, grafische Aktualisierungen aus Hintergrundprozessen sicher anzustoßen.

Zur übersichtlichen Darstellung der MQTT-Nachrichten sollte die GUI-Bibliothek Tabellenansichten und hierarchische Strukturen einfach implementieren lassen. Dabei ist eine automatisierte Aktualisierung bei Datenänderung oder neuen Daten ausschlaggebend.

Bei C# stehen verschiedene Technologien zur Verfügung, darunter insbesondere die bereits erwähnte Windows Presentation Foundation (WPF), WinUI sowie .NET MAUI [15]. Diese Frameworks unterscheiden sich vor allem hinsichtlich ihres Plattformumfangs, Reifegrades sowie ihrer Ausrichtung auf zukünftige Anwendungsbereiche.

Jede der drei Technologien bietet eine Abstraktionsschicht zwischen der Anwendungslogik und der grafischen Darstellung. Dadurch wird eine klare Trennung von Darstellung und Geschäftslogik ermöglicht [15]. Insbesondere durch die Verwendung von XAML wird die deklarative Beschreibung von Benutzeroberflächen unterstützt, was die Strukturierung und Wartbarkeit von Anwendungen verbessert.

WPF stellt eine etablierte und weit verbreitete Technologie für die Entwicklung von Desktop-Anwendungen unter Windows dar. Durch ein integriertes Event-System sowie einen zentralen Eventloop eignet sich WPF besonders für ereignisgesteuerte Anwendungen. Zudem bietet das Framework eine umfangreiche Unterstützung für Datenbindung, Styles sowie das MVVM-Architekturmuster,

wodurch datengetriebene Benutzeroberflächen effizient umgesetzt werden können [15]. Insbesondere bei Anwendungen wie Monitoring-Systemen, bei denen kontinuierlich neue Daten wie zum Beispiel Nachrichten verarbeitet und dargestellt werden müssen, spielt diese Eigenschaft eine zentrale Rolle.

WinUI ist eine modernere Weiterentwicklung im Windows-Ökosystem und ist Teil des Windows App SDK. Es basiert auf ähnlichen Konzepten wie WPF, insbesondere der Nutzung von XAML, setzt jedoch stärker auf moderne UI-Designs. Durch eine engere Integration in die aktuellen Windows-Versionen empfiehlt Microsoft WinUI für neue native Windows-Anwendungen [15].

Im Gegensatz dazu verfolgt .NET MAUI einen plattformübergreifenden Ansatz. Es ermöglicht die Entwicklung von Anwendungen für Windows, macOS, iOS und Android auf Basis einer gemeinsamen Codebasis. Dadurch ist MAUI insbesondere für Projekte geeignet, bei denen eine breite Plattformunterstützung erforderlich ist. Im Gegensatz zu WPF ist dieses Framework weniger ausgereift, da es sich noch in aktiver Weiterentwicklung befindet. Dadurch kann es in bestimmten Szenarien noch zu Einschränkungen hinsichtlich Stabilität und Tooling kommen [15].

Der zentrale Unterschied zwischen diesen Technologien liegt in ihrer Zielsetzung. Während WPF den Fokus auf eine stabile und leistungsfähige Windows-Desktopanwendung legt, legt WinUI den Fokus auf moderne Grafik und richtet sich auf die Zukunft aus. MAUI hingegen verfolgt das Ziel, plattformübergreifende Anwendungen zu schaffen [15].

Für die Umsetzung dieser Arbeit bietet WPF aufgrund seiner hohen Reife, stabilen Architektur sowie der sehr guten Unterstützung datengetriebener, ereignisbasierter Benutzeroberflächen eine besonders geeignete Grundlage. Durch die langjährige Etablierung des Frameworks sowie die große Community stehen umfangreiche Dokumentationen und Unterstützungsmöglichkeiten zur Verfügung.

Obwohl WPF aufgrund seines hohen Reifegrades und seiner umfassenden Unterstützung verschiedener Funktionen ausgewählt wurde, stellt die Beschränkung auf das Windows-Betriebssystem einen Nachteil dar. Eine spätere Unterstützung von Linux oder macOS würde eine Umstellung auf MAUI oder andere plattformübergreifende Technologien erforderlich machen.

5.2 Anforderungen an die Test- und Entwicklungsumgebung

5.2.1 Entwicklungsumgebung

Für die integrierte Entwicklungsumgebung (IDE) für dieses Projekt gibt es einige Anforderungen, denen sie gerecht werden müssen. Die IDE muss die Programmiersprache C#, das .NET-Ökosystem sowie XAML zur GUI-Entwicklung unterstützen. Sie muss die Möglichkeit bereitstellen, Desktop-Anwendungen unter Windows zu entwickeln. Um Fehler leichter zu finden, ist auch das Debugging von asynchronem Code und nebenläufigen Prozessen gefordert. Außerdem wird eine Abhängigkeitsverwaltung sowie eine Projekt- und Buildverwaltung benötigt.

Gängige Entwicklungsumgebungen sind Visual Studio 2022, Visual Studio Code oder JetBrains Rider. Diese unterscheiden sich vor allem durch den Umfang der GUI-Entwicklungshilfen, die Unterstützung von XAML, das Debugging-Verhalten sowie die Benutzerfreundlichkeit.

Visual Studio 2022 bietet eine vollwertige IDE, die speziell für C# und .NET entwickelt wurde. Es ist die offizielle Entwicklungsumgebung von Microsoft und hat daher eine sehr gute Integration von WPF, WinUI und XAML. Benutzeroberflächen lassen sich durch einen grafischen Designer schon während der Programmierung anzeigen, ohne dafür das Projekt bauen zu müssen. Das erleichtert die XAML-Programmierung erheblich. Durch die Unterstützung von Drag and Drop können kleine Anwendungen sogar ohne Kenntnisse über XAML erstellt werden. Neben dem leistungsfähigen Debugging inklusive Threads, async und Tasks liefert Visual Studio integrierte Tools für die Paketverwaltung, Build-Prozesse und Projektverwaltung. Visual Studio 2022 eignet sich daher besonders für die Entwicklung von Windows-Desktop-Anwendungen und bietet durch seine große Community eine ausführliche Dokumentation [38].

JetBrains Rider bietet ebenfalls eine gute Unterstützung für C#- und .NET-Programmierung. Es stellt starke Code-Analyse- und Refactoring-Funktionen bereit. Im Gegensatz zu Visual Studio 2022 bietet es keinen vollständig integrierten visuellen XAML-Designer. Dadurch ist zwar die GUI-Entwicklung möglich, aber weniger komfortabel als in Visual Studio. Insgesamt ist JetBrains Rider besonders für große Projekte und erfahrene Entwickler geeignet [26].

Visual Studio Code gilt als leichtgewichtiger Editor und ist somit keine vollständige IDE. Es lässt sich zwar durch Plugins wie zum Beispiel C#-Extensions fast unendlich erweitern, bietet aber keinen grafischen XAML-Designer, wodurch die GUI-Entwicklung komplex wird. Visual Studio

Code legt den Fokus eher auf die Webentwicklung und auf Skripte, weswegen es zur Entwicklung von Desktop-Anwendungen nur eingeschränkt geeignet ist [41].

Insgesamt zeigt die Analyse, dass Visual Studio 2022 alle definierten Anforderungen an eine IDE vollständig erfüllt. Dabei ist die nahtlose Integration in das *.NET*-Ökosystem und die umfassende Unterstützung von C# und XAML besonders hervorzuheben. Es bietet eine besonders geeignete Kombination aus Funktionsumfang, Benutzerfreundlichkeit und Stabilität, weswegen es in dieser Arbeit zur Entwicklung der Monitoring-Applikation verwendet wird.

5.2.2 Testumgebung

Für die Entwicklung und Validierung der Applikation ist eine reproduzierbare Testumgebung erforderlich. Diese muss es ermöglichen, reale MQTT-Kommunikation von produktiven Systemen zu simulieren und zu analysieren. Dabei sollte die Testumgebung einen MQTT-Broker bereitstellen und reale Topics und Nachrichtenlasten unterstützen.

Daneben wird eine Möglichkeit zur Erzeugung hoher Nachrichtenfrequenzen benötigt, damit die Reaktionsfähigkeit und das Zusammenspiel der verschiedenen Programmebenen analysiert werden kann. Dadurch können die Leistungsgrenzen der Applikation ermittelt werden.

6 Technisches Konzept

6.1 Anforderungen an die Applikation

6.1.1 Nachrichtendarstellung und Filterung

Ein zentraler Bestandteil der Applikation ist die nachträgliche Analyse von MQTT-Nachrichten. Im Gegensatz zu bestehenden Tools wie MQTT.fx soll die Anwendung die Möglichkeit bieten, bereits empfangene Nachrichten im Nachhinein zu filtern. Die Filterung erfolgt dabei auf Basis von Topic-Präfixen. Ergänzend dazu wird eine Volltextsuche benötigt, die eine schnelle Navigation innerhalb großer Datenmengen ermöglicht. Hierbei sollen sowohl die Payload, das Topic als auch der Zeitstempel durchsucht werden können. Ziel ist eine effiziente Analyse auch bei stark ansteigenden Nachrichtenvolumina.

Ein weiterer funktionaler Bestandteil ist die parallele Analyse mehrerer Datenströme. Hierzu soll die Applikation die gleichzeitige Anzeige mehrerer Log-Fenster ermöglichen. Für jedes dieser Fenster können individuelle Filter gesetzt sowie die dargestellten Inhalte separat exportiert werden. Dadurch wird eine flexible und unabhängige Auswertung verschiedener Topics ermöglicht.

6.1.2 Automatische Fehlererkennung

Die Anwendung soll darüber hinaus Funktionen zur automatischen Fehlererkennung bereitstellen. Dazu gehört die Validierung der Payloads auf korrektes JSON-Format sowie die Erkennung struktureller Abweichungen. Ein Beispiel für eine strukturelle Abweichung ist das Fehlen des sogenannten Headers im Payload, der in jeder Nachricht den Sender und weitere Nachrichtendetails wie den Nachrichtentyp beschreibt. Zusätzlich sollen auch Fehler innerhalb von Maschinenprozessen

anhand der Nachrichten erkannt werden. Dazu werden die beim Prozess versendeten Nachrichten mit vordefinierten erwarteten Nachrichten verglichen. Sollte hier eine Abweichung erkannt werden, soll die Anwendung eine entsprechende Fehlernachricht anzeigen. Ziel ist eine frühzeitige Identifikation fehlerhafter oder inkonsistenter Nachrichten. Ergänzend dazu muss eine Überwachung der Teilnehmer erfolgen, um festzustellen, ob alle Systemkomponenten mit dem Broker verbunden sind und ihre Verbindung aufrechterhalten.

6.1.3 Überwachung der Automatisierungs-Abläufe

In einem zusätzlichen Reiter sollen abgeschlossene und laufende Abläufe der Verpackungslinie eingesehen werden können. In den einzelnen Automatisierungs-Abläufen werden die erwarteten und bereits empfangenen Nachrichten dargestellt. Sollte ein neuer Prozess beginnen, während noch nicht alle erwarteten Nachrichten eingingen, wird der Prozess mit einer Fehlermeldung abgespeichert und angezeigt.

Zu den Abläufen gehören Linienstart und -stopp, Quittierung von Fehlermeldungen, Leerfahren der Linie, Artikelwechsel, fliegender Artikelwechsel und Batchhandlingprozesse. Dabei können die Abläufe Fehlerquittierung, Linienstart und Linienstopp nicht parallel laufen und allgemein kein Linienablauf gleichzeitig mehrfach aktiv sein. Dadurch können fehlerhaft abgeschlossene Abläufe erkannt werden.

6.1.4 Simulationsmöglichkeiten

Eine weitere Anforderung stellt die erweiterte Publish-Funktionalität dar. Nachrichten sind manuell erstellbar und können unter einem bestimmten Topic gesendet werden. Dabei ist vorgesehen, häufig genutzte Nachrichten als Vorlagen zu speichern und wiederzuverwenden. Zusätzlich soll die Möglichkeit bestehen, Nachrichten in festgelegten Zeitintervallen automatisch zu senden. Dies unterstützt insbesondere Test- und Simulationsszenarien.

Ergänzend dazu ist eine Simulationsfunktion vorgesehen, welche das Senden vordefinierter Nachrichten über eine direkte Benutzerinteraktion ermöglicht. Dabei soll die Initiierung solcher Aktionen durch konfigurierbare Bedienelemente erfolgen. Ziel ist eine vereinfachte Durchführung von Tests sowie der Erzeugung reproduzierbarer Datenströme.

6.1.5 Anforderungen an die Performance

Neben den funktionalen Anforderungen spielt die Performance der Anwendung eine entscheidende Rolle. Dies ist insbesondere bei hohen Nachrichtenfrequenzen in industriellen Anlagen relevant. Die Verarbeitung und Darstellung der Nachrichten muss so gestaltet sein, dass auch bei hohen Datenraten eine flüssige Bedienung gewährleistet bleibt. Gleichzeitig ist eine skalierbare Architektur notwendig, um zukünftige Erweiterungen des Funktionsumfangs zu ermöglichen.

Abschließend soll die Umsetzung dieser Anforderungen in einer übersichtlichen und intuitiv bedienbaren Benutzeroberfläche erfolgen. Auch bei komplexen Analyseprozessen muss eine schnelle Orientierung sowie eine effiziente Interaktion gewährleistet sein. Die übergeordnete Zielsetzung besteht darin, bestehende Werkzeuge in den Bereichen Datenanalyse, Fehlersuche und Testautomation funktional zu erweitern und insbesondere hinsichtlich Benutzerfreundlichkeit und Analysefähigkeit zu verbessern.

6.1.6 Erweiterbarkeit und Zukunftssicherheit

Die Architektur soll so gestaltet werden, dass sie nicht nur für den aktuellen Zweck geeignet ist, sondern auch langfristig wartbar und erweiterbar bleibt. Insbesondere in der Industrie und bei der Kommunikation mit MQTT-Systemen ändern sich die Anforderungen oft im Laufe der Zeit. Neue Maschinen, zusätzliche Linien, neue Nachrichtentypen oder geänderte Datenstrukturen sind Beispiele dafür, auf die ein Überwachungstool flexibel reagieren muss.

Ein wichtiger Aspekt der Erweiterbarkeit besteht darin, fachliche Regeln und Prüfmechanismen nicht direkt im Programmcode zu verankern, sondern sie über Konfigurationsdateien abzubilden. In diesem Konzept wird dies durch den Einsatz externer JSON-Dateien umgesetzt. Sowohl die Teilnehmerüberprüfung als auch die optionale Schema-Validierung sollen auf Konfigurationsdateien basieren, die zur Laufzeit geladen werden können. Dadurch können neue Teilnehmer, Maschinen oder Validierungsregeln hinzugefügt werden, ohne dass die Applikation angepasst oder neu kompiliert werden muss.

Die optionale Schema-Validierung ist ein wichtiger Baustein für die Zukunftssicherheit. Durch die Nutzung versionierter Nachrichtenschemas können verschiedene Nachrichtenformate parallel unterstützt werden. In realen Produktionsumgebungen ist es selten der Fall, dass alle Systeme gleichzeitig auf eine neue Software- oder Protokollversion aktualisiert werden. Die Fähigkeit, meh-

rere Schema-Versionen zu erkennen und zu validieren, ermöglicht einen schrittweisen Übergang zwischen verschiedenen Versionsständen. Alte Systeme bleiben weiterhin analysierbar, während neue Systeme bereits mit erweiterten oder geänderten Datenstrukturen arbeiten können.

Ein weiterer wichtiger Faktor für die Erweiterbarkeit ist die lose Kopplung der einzelnen Schichten. Die Netzwerkschicht ist ausschließlich für die MQTT-Kommunikation zuständig und hat kein Wissen über die interne Darstellung oder die Analyseergebnisse der Nachrichten. Ebenso ist die grafische Oberfläche von der konkreten Netzwerktechnik entkoppelt und arbeitet ausschließlich mit Datenmodellen und Analyseergebnissen. Diese Trennung ermöglicht es, einzelne Komponenten auszutauschen oder zu erweitern, ohne unbeabsichtigte Auswirkungen auf andere Teile der Applikation zu verursachen. Diese lose Kopplung wird durch die Verwendung von Interfaces sowie klar definierten Service-Schichten umgesetzt.

Auch die Analyse- und Validierungslogik soll bewusst modular aufgebaut werden. Die mehrstufige Verarbeitung ermöglicht es, zusätzliche Prüfungen schrittweise zu ergänzen. Neben den bereits vorgesehenen Basis- und Konsistenzprüfungen können zukünftig weitere Analysearten integriert werden. Denkbar sind beispielsweise zeitbasierte Prüfungen, bei denen Nachrichten auf erwartete Wiederholungsintervalle oder Heartbeat-Mechanismen überprüft werden.

Die Nutzung eines datengetriebenen GUI-Konzepts trägt ebenfalls wesentlich zur Zukunftssicherheit bei. Da die Benutzeroberfläche nicht direkt durch Programmcode manipuliert wird, sondern sich automatisch aus den zugrunde liegenden Datenmodellen aktualisiert, kann das grafische Erscheinungsbild flexibel erweitert werden. Neue Ansichten, Filterfunktionen oder Darstellungsformen lassen sich hinzufügen, ohne die zugrundeliegende Analyse-Logik anpassen zu müssen.

Ein weiterer Aspekt der Zukunftssicherheit ist die Skalierbarkeit hinsichtlich der Nachrichtenlast. Ein zentraler Bestandteil ist dabei die Nutzung von asynchronen Programmier Techniken (async/await), die eine nicht-blockierende Verarbeitung von Nachrichten ermöglicht.

Abschließend lässt sich festhalten, dass die konzipierte Architektur bewusst auf langfristige Nutzung und Erweiterbarkeit ausgelegt sein soll. Durch konfigurationsgetriebene Validierungsmechanismen, versionierte Nachrichtenschemas, lose Kopplung der Systembestandteile sowie eine datengetriebene Benutzeroberfläche soll eine hohe Flexibilität erreicht werden. Diese Eigenschaften machen das Überwachungstool nicht nur für den aktuellen Entwicklungsstand praktikabel, sondern erlauben auch eine zukünftige Anpassung an neue Anforderungen, ohne eine grundlegende Neugestaltung der Applikation erfordern zu müssen. Neben diesen Vorteilen bringt diese Anforderung einen wesentlich erhöhten Programmieraufwand mit sich, der in einer Projektarbeit teilweise

nicht umsetzbar sein kann.

6.2 Prototypische Umsetzung und Erkenntnisse aus der Entwicklung

6.2.1 Umsetzung des Prototyps in Python

Zum Testen der Realisierbarkeit der Applikation und ihrer Anforderungen wird ein Prototyp eines ersten funktionalen Systems zur Überwachung von MQTT-Nachrichten entwickelt. Dabei liegt der Fokus zunächst auf dem Empfang und der Darstellung der Nachrichten, der Analyse ihrer Inhalte sowie der damit verbundenen Erkennung einfacher Fehler. Die Umsetzung ist bewusst minimal gehalten, um nur Grundfunktionalitäten zu testen.

Für die prototypische Umsetzung wird Python in Kombination mit dem GUI-Framework Tkinter verwendet. Dieses Framework gilt als sehr leichtgewichtig und vereinfacht die Erstellung des GUI stark [34]. Bereits der Prototyp weist einen modularen Aufbau auf, indem das Programm in mehrere, nach Aufgaben gegliederte Dateien aufgeteilt ist.

Für die Architektur des Prototyps existiert eine zentrale Klasse „MQTTGui“. Diese ist für die Erstellung der grafischen Oberfläche, aber auch für die Datenhaltung über die Arrays `Belf.messages` und `Belf.topics` zuständig. Die GUI besteht aus einer Log-Anzeige, Fehleranalyse und der Verbindungssteuerung.

6.2.2 Verbindungsaufbau und Nachrichtenempfang

Der Verbindungsaufbau wird in der Subscriber-Klasse implementiert. Dazu wird ein Objekt der Klasse *MQTT_Subscriber* erstellt und diesem die in der GUI eingegebenen Broker-IP und Port übergeben. Dadurch verbindet sich das Programm mit dem Broker und abonniert alle Topics. Bei der Erstellung des Objekts wird auch eine Callbackfunktion übergeben, die immer aufgerufen wird, wenn neue Nachrichten eintreffen (Listing: 6.1).

Listing 6.1: Callbackfunktion für den Message-Empfang

```
1 def on_message(self, topic, payload, qos, retained):
2     ts = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
3     self.messages.append((ts, retained, topic, payload, qos))
4
5     self.tool_pane.analyze_messages(self.messages, only_new=True)
6
7     if self.save_var.get():
8         log_to_file(topic, payload, qos, retained)
9
```

In dieser Callback-Funktion wird jede Nachricht in „self.messages“ mit Zeitstempel, dem Topic, Payload und dem QoS gespeichert und an die Fehleranalyse-Komponente übergeben. Optional können die Nachrichten auch in einer externen Datei gespeichert werden.

6.2.3 Nachrichtenverarbeitung

Die Darstellung der Nachrichten in einem oder in verschiedenen Nachrichtenfenster implementiert die Klasse „LogPane“. In dieser wird neben Autoscroll-Funktionen auch eine Filterung nach Topics erstellt, die man in der GUI für jedes Nachrichtenfenster einzeln anwenden kann. Damit nur die Nachrichten der einzelnen Topics angezeigt werden, wird in der Methode *append* herausgelesen, was in dem Filter eingegeben ist, und entscheidet auf dieser Grundlage über das Einfügen der einzelnen Nachrichten in die jeweilige Nachrichtenliste (Listing: 6.2).

Listing 6.2: gefilterte Anzeige von Nachrichten

```
1 def append(self):
2     f = self.filter.get()
3     for ts, retained, topic, payload, qos in self.messages[self.last_index:]:
4         if f == "ALL" or topic.startswith(f):
5             self._insert(ts, retained, topic, payload, qos)
6     self.last_index = len(self.messages)
7
```

6.2.4 Umgang mit großen Datenmengen

Als weitere Funktion werden sehr große Payloads, die für die Anzeige die GUI stark belasten, ausgelagert. Durch einen Klick auf die Nachricht wird diese in einem externen Editor geöffnet.

Ähnlich funktioniert auch das Aufrufen der Nachricht durch das Anklicken der Fehlermeldung in der Analysekomponente. In diesen Fehlermeldungen werden jeweils die Nachrichten- sowie Fehlernummern gespeichert. Beim Anzeigen der Fehlermeldung wird der Nachricht ein sogenannter *Tag* übergeben. Dieser enthält die Fehlernummer und die Methode, die abgearbeitet werden soll, wenn auf die Nachricht geklickt wird. In diesem Fall wird die Methode *on_error_click* aufgerufen (Listing: 6.3).

Listing 6.3: gefilterte Anzeige von Nachrichten

```
1 def on_error_click(self, index):
2     if not self.logpane:
3         return
4     self.gui.show_logs()
5     error = self.errors[index]
6     msg_id = error.get("msg_id")
7     if msg_id is None:
8         return
9
10    self.logpane.jump_to_message(msg_id)
11
```

Für die fertige Applikation darf eine sehr große Datenmenge kein Problem darstellen. Deswegen ist es wichtig, dass von Anfang an großen Wert auf die Performance und Stabilität des Programms gelegt wird.

Durch die Implementierung einer separaten Analysekomponente können die Nachrichten auf das Vorhandensein eines Topics und Payloads überprüft werden. Des Weiteren werden die Payloads daraufhin auf ein korrektes JSON-Format überprüft. Als Zusatzfunktion dieser Analysekomponente kann eine sogenannte *Line-Config*-Datei aus den Dateien heraus geladen werden. Diese beinhaltet alle Teilnehmer mit (Host-)Namen im JSON-Format. So kann überprüft werden, ob sich alle Teilnehmer beim Broker registrierten und auch weiterhin die Verbindung zum Broker nicht wieder schließen.

Somit enthält bereits der Prototyp einige Anforderungen an die Applikation und kann bereits für einfache Analysen eingesetzt werden.

6.2.5 Grenzen des Prototyps

Im Rahmen der prototypischen Umsetzung mit der verwendeten Struktur wurden klare Grenzen identifiziert. Insbesondere die grafische Darstellung in Python kommt bei zunehmender Komplexität an ihre Grenzen. Vor allem umfangreiche Datenbindung, visuell komplexe Tabellenansichten oder eine tiefe Integration mit dem Betriebssystem lassen sich in dem Python-GUI-Framework nur begrenzt elegant umsetzen. Erweiterungen oder individuelle Anpassungen führen oft zu hohem Implementierungsaufwand und schlechter Wartbarkeit. Dies ist jedoch nicht ausschließlich auf die Programmiersprache Python zurückzuführen, sondern liegt auch an der geringen Modularisierung und Performance-Auslegung des Programms und ist teilweise auf die gewählte Architektur zurückzuführen. Dadurch wird aber deutlich, dass für die Programmierung in C# von Anfang an stark modularisiert werden muss, damit das Programm langfristig erweiterbar und leicht anpassbar bleibt. Des Weiteren muss in C# eine asynchrone Performance-orientierte Verarbeitung der Nachrichten ermöglicht werden, damit diese die UI nicht blockieren und das Programm aufgrund von zu großen Datenmengen abstürzt.

Die interpretierte Programmiersprache Python weist im Vergleich zu kompilierten Sprachen wie C# eine geringere Laufzeitperformance auf [5]. Besonders bei hohen Nachrichtenraten oder großen Datenmengen kann es zu Verzögerungen in der Verarbeitung und Darstellung kommen. Dies spielt gerade bei der Speicherung der Nachrichten in externe Text-Dateien oder Datenbanken eine große Rolle, da dieser Vorgang mehr Zeit beansprucht. Für solche Prozesse bietet Python dafür die Bibliothek „asyncio“, mit der asynchrone Prozesse effizient verwaltet und ereignisgesteuert ausgeführt werden können. Diese wurde jedoch in dem Prototyp nicht verwendet, weswegen die Performance verschlechtert wird [3].

Für eine produktive Anwendung bestehen jedoch klare Anforderungen an Performance, Wartbarkeit, Typensicherheit und Integration. Diese Anforderungen sprechen deutlich für eine Neuimplementierung in C#. Diese Programmiersprache bietet eine bessere Unterstützung für asynchrone Verarbeitungen, moderne GUI-Frameworks wie WPF und ermöglicht eine stärkere Integration in Windows-Systeme [5]. Dadurch wird eine langfristig stabile, erweiterbare und performante Lösung ermöglicht.

Die gewonnenen Erkenntnisse aus dem Prototypen fließen direkt in die Konzeption der finalen Applikation ein.

6.3 Konzeptplan und Umsetzung der Benutzeroberfläche

6.3.1 Systemarchitektur

Als Architekturgrundstein für die Applikation wird die MVVM (Model-View-ViewModel)-Architektur genutzt. Durch diese Architektur entsteht eine saubere Trennung von UI und Logik. Die View dient dabei ausschließlich der Darstellung, während das View-Model die Logik und Bereitstellung der Daten für das View beinhaltet. Im UI-Programmcode selbst existiert dabei bestenfalls keinerlei Logik. Die Bereitstellung der Daten erfolgt durch die Models.

Durch dieses Modell entsteht wenig Code für die UI, da diese lediglich sogenannte Bindings hinterlegt, die von den View-Models gesteuert werden. So können die View-Models und Models auch ohne UI getestet werden und die View jederzeit einfacher ersetzt werden.

In diesem Projekt werden die View-Models durch Service- und Repository-Dateien unterstützt, in denen die Anwendungslogik verankert ist. Dadurch übernimmt das View-Model ausschließlich die Interaktion mit der Benutzeroberfläche und die Bereitstellung der Daten. Die Repository-Dateien übernehmen die Aufgabe der Initialisierung der Tabellen in der Datenbank sowie Speicherung und Laden von Daten aus der Datenbank. Die Service-Dateien verarbeiten die Daten der View-Models und rufen wenn nötig die Methoden der Repositorien auf.

Die Verbindung zwischen View und View-Model erfolgt über das Data-Binding-System von WPF. Dabei wird dem DataContext einer View ein entsprechendes ViewModel zugewiesen. Die Benutzeroberfläche kann anschließend über Bindings auf die Eigenschaften und Befehle des View-Models zugreifen, ohne dieses direkt zu kennen. Dadurch entsteht eine lose Kopplung zwischen Darstellung und Anwendungslogik [17].

Änderungen an Eigenschaften des View-Models werden über das Interface `INotifyPropertyChanged` an die View weitergegeben. Dadurch werden zum einen Änderungen aus der View automatisch in die Variablen der View-Model geschrieben, zum anderen Änderungen aus dem View-Model automatisch in der Benutzeroberfläche dargestellt. Für Auflistungen wird eine `ObservableCollection` verwendet, die Änderungen innerhalb einer Sammlung selbständig an die View weitergibt. Benutzerinteraktionen wie beispielsweise das Betätigen eines Buttons oder die Auswahl eines Tabelleneintrags können über Commands an das View-Model weitergeleitet werden. Die eigentliche Verarbeitungslogik verbleibt dabei im View-Model, während die View ausschließlich für die Dar-

stellung und Entgegennahme von Benutzereingaben zuständig ist [17].

Zur erleichterten Umsetzung der MVVM-Struktur wird die „CommunityToolkit.Mvvm“-Bibliothek verwendet. Mit dieser Bibliothek kann zum einen die ansonsten aufwendige Implementierung des Interfaces `INotifyPropertyChanged` erleichtert werden. Mittels Source-Generatoren wird die benötigte Property-Changed-Logik automatisch erzeugt. Dadurch kann redundanter Boilerplate-Code vermieden werden, was sowohl den Implementierungsaufwand als auch die Lesbarkeit des Quellcodes verbessert [20, S. 106 ff.].

Darüber hinaus ermöglicht die Bibliothek `CommunityToolkit.Mvvm` eine einfache Implementierung von `ICommand`-Schnittstellen. Diese werden benötigt, um Benutzerinteraktionen wie beispielsweise Button-Klicks an das ViewModel weiterzuleiten. Während ohne die Bibliothek zusätzlicher Programmieraufwand für die Implementierung eines Commands erforderlich ist, genügt mit `CommunityToolkit.Mvvm` die Verwendung des Attributs `[RelayCommand]` oberhalb der entsprechenden Methode. Aus dieser Methode wird automatisch ein Command erzeugt, welches anschließend über ein Binding in der View verwendet werden kann [20, S. 106 ff.].

6.3.2 Datenbankkonzept

Ziel des Datenbank-Designs ist die Speicherung aller MQTT-Nachrichten, eine effiziente Suche in allen Nachrichten, die Teilnehmer und Statusüberwachung, eine Prozessanalyse und die Speicherung von Vorlagen und Einstellungen. Dazu werden unterschiedliche Haupttabellen erstellt.

Die Tabelle *Messages* speichert die ID, Zeitstempel, Topic, Payload und den Sender der Nachricht. Die Tabelle *Participants* speichert die Namen und den Maschinentyp aller Teilnehmer sowie deren zuletzt gesehener Zeitstempel, den Status und den Zustand, in dem sie sich gerade befinden. In der Tabelle *Errors* werden alle Fehler mit der entsprechenden Nachrichten-ID sowie Fehlertext und -Art gespeichert. In der Tabelle *ProcessEvent* werden die verschiedenen durchlaufenen Prozesse sowie deren Startzeit, Endzeit, bereits empfangene Nachrichten und die als Nächstes erwarteten Nachrichten gespeichert.

Die bereits beschriebenen Tabellen sollen ihren Inhalt beim Starten der Anwendung und bei der manuellen Verbindungstrennung zum Broker wieder löschen, damit die Nachrichten, Teilnehmer und Prozesse bei einer neuen Verpackungslinie auch wieder neu aufgezeichnet werden können. Die folgenden Tabellen besitzen dieses Verhalten gezielt nicht, da diese Informationen zwischen

verschiedenen Anwendungssitzungen gespeichert werden sollen.

In den Tabellen *TemplatesConnection* und *TemplatesPublishingMessages* werden Vorlagen für Verbindungen sowie für zu versendende Nachrichten gespeichert. Dadurch können gespeicherte Vorlagen per Mausklick für den Verbindungsaufbau oder das Senden von Nachrichten verwendet werden. In Settings werden alle Einstellungen gespeichert, damit diese auch bei einem Neustart der Anwendung noch gespeichert sind.

6.3.3 Nachrichtenfluss und -Verarbeitung

Beim Eintreffen einer Nachricht soll eine Callback-Funktion über ein Event aufgerufen werden. Diese übernimmt dann zum einen die Aktualisierung der UI, ruft zum anderen aber auch einen Service auf, der sich der Verarbeitung der Nachricht widmet. In Abbildung B.1 ist der Ablauf für diesen Service visualisiert.

Zuerst wird die Payload der Nachricht in das JSON-Format überführt. Dadurch lässt sich der Sender der Nachricht aus der Payload herauslesen. Topic, Zeitstempel, Sender und Payload werden daraufhin in der Datenbank abgespeichert. Bei der JSON-Formatierung wird sofort auf Richtigkeit des Formats und dem Vorhandensein eines sogenannten Headers geprüft, der bei allen Nachrichten vorhanden sein muss. Sollte diese Prüfung fehlschlagen, wird der Fehler zusammen mit einer Fehlernachricht und der Identifikationsnummer der Nachricht in der Fehlertabelle der Datenbank gespeichert.

Als nächstes erfolgt die Aktualisierung des Senders in der Teilnehmerliste. Sollte der Sender noch nicht existieren, wird ein neuer Teilnehmer in die Liste aufgenommen und eine entsprechende Warnung in der Fehlerliste erzeugt.

Als letztes wird die Nachricht auf Prozesse analysiert. Dies startet mit der Überprüfung auf einen neuen Prozess. Wenn dies zutrifft, wird ein neuer Prozess in die ProcessEvent-Tabelle der Datenbank geschrieben sowie die erwarteten Nachrichten aus der Teilnehmerliste und anhand des Prozesstyps abgeleitet. Gehört die Nachricht zu einem laufenden Prozess, wird sie auf Richtigkeit überprüft. Ist sie fehlerhaft, wird ein Fehler mit Fehlernachricht und der Nachricht-ID in die Fehlertabelle gespeichert. Wenn die Nachricht einer der erwarteten Nachrichten entspricht, werden die erwarteten und bereits empfangenen Nachrichten in der Datenbank aktualisiert. Endet der Prozess durch eine Nachricht, wird dem Prozess in der Datenbank ein End-Zeitstempel hinzugefügt und

als abgeschlossen gekennzeichnet.

Diesen Ablauf durchläuft jede eintreffende MQTT-Nachricht.

6.3.4 Performance-Konzept

Damit die UI trotz vieler Nachrichten flüssig bedienbar bleibt und die Anwendung nicht zu viel Arbeitsspeicher in Anspruch nimmt, wird ein Threading- und Paging-Konzept erstellt.

Während die GUI im Haupt-Thread ausgeführt wird, erfolgt der MQTT-Nachrichtenempfang und die Datenverarbeitung in einem beziehungsweise mehreren Background-Thread. Datenbankzugriffe, Formatparsing und Überprüfungen werden asynchron verarbeitet, damit auch weitere Nachrichten empfangen werden können. Das View-Model stellt daraufhin alle Daten für die View bereit.

Neben einem Threading-Konzept wird auch ein Paging-Konzept in Betracht gezogen. Dieses dient der Verringerung des Arbeitsspeicher-Verbrauchs bei Langzeit-Aufnahmen der Linienkommunikation. Dabei werden alle Nachrichten mit Fehlermeldungen und weiteren Informationen in einer Datenbank gespeichert. Im Programm selbst wird dann nur eine gewisse Anzahl an Nachrichten aus der Datenbank in eine lokale Liste geladen, die gerade benötigt werden. Das bedeutet ein automatisches Laden älterer oder neuerer Nachrichten, sobald in der Nachrichtenliste geblättert wird [33].

Das vorgestellte Paging-Konzept reduziert den Arbeitsspeicherverbrauch erheblich. Gleichzeitig entsteht jedoch zusätzlicher Aufwand aufgrund von häufigen Datenbankzugriffen beim Navigieren durch große Nachrichtenmengen. Die tatsächliche Effizienz dieses Ansatzes muss daher durch Performancetests validiert werden.

6.3.5 Aufbau der Benutzeroberfläche

Grundlegend soll die GUI so wie in Abbildung 6.1 gezeigt aufgebaut werden. Links soll eine Seitenleiste angezeigt werden, in der man zwischen den Reitern wechseln kann. In einer Statusleiste im oberen Bereich sollen ständig die Teilnehmer mit ihrem Zustand und aktuellen Prozess angezeigt

werden, während eine weitere Statusleiste unten allgemeine Informationen bereitstellt. Diese Informationen beinhalten die Anzahl empfangener Nachrichten, den Verbindungsstatus zum Broker sowie interne Meldungen der Anwendung.

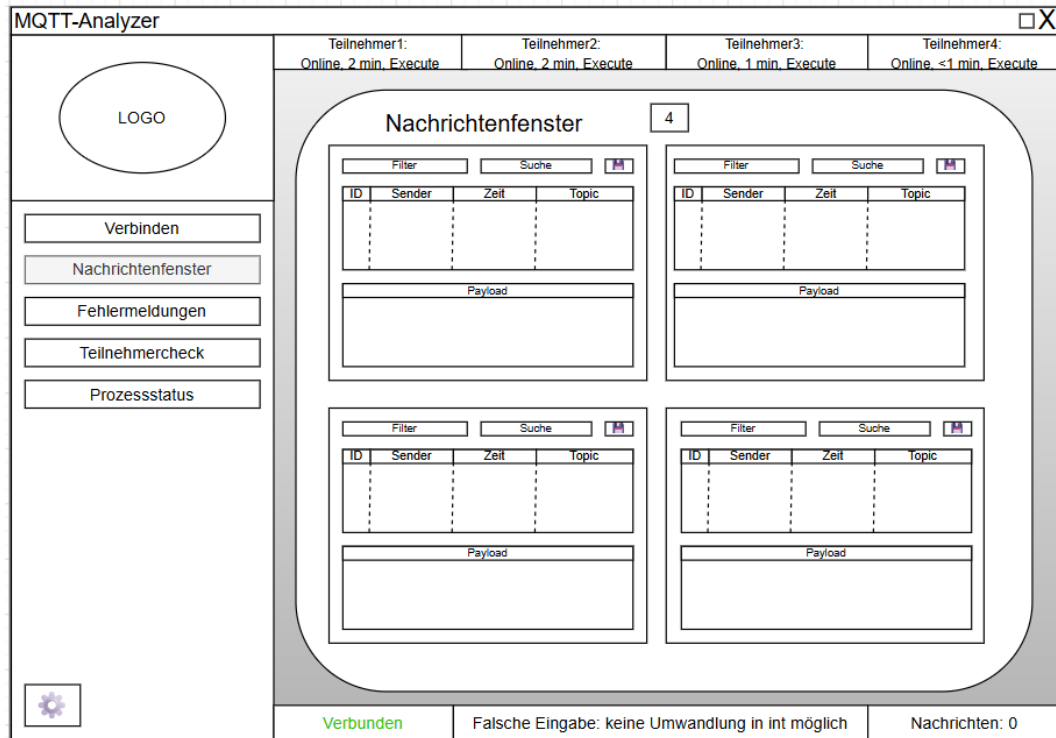


Abbildung 6.1: Grundlegendes Konzept für die GUI

7 Umsetzung

7.1 Realisierung der Softwarearchitektur

Für die Umsetzung der Anwendung wurde die in Kapitel 6 entwickelte MVVM-Architektur verwendet. Während die Benutzeroberfläche ausschließlich für Darstellung der Informationen verantwortlich ist, erfolgt die Verarbeitung der MQTT-Nachrichten vollständig innerhalb separater Service-Komponenten. Die Datenhaltung wurde in Repository-Klassen ausgelagert, welche den Zugriff auf die SQLite-Datenbank kapseln. Dadurch konnte eine klare Trennung zwischen Darstellung, Geschäftslogik und Persistenz erreicht werden.

Im Verlauf der Implementierung zeigte sich insbesondere bei der Verarbeitung großer Nachrichtmengen die Bedeutung dieser Trennung. Analysefunktionen konnten erweitert werden, ohne Änderungen an der Benutzeroberfläche vornehmen zu müssen. Gleichzeitig war es möglich, einzelne Komponenten unabhängig voneinander zu testen.

Zur Verringerung des Implementierungsaufwands wurde das `CommunityToolkit.MVVM` eingesetzt. Dadurch konnten Eigenschaften und Befehle automatisiert erzeugt werden, wodurch sich der manuell zu implementierende Programmcode deutlich reduzierte.

7.2 Verarbeitung eingehender MQTT-Nachrichten

Der Empfang der MQTT-Nachrichten sowie auch der Verbindungsaufbau, Verbindungsabbau und Erkennung von Verbindungsunterbrechungen wurde innerhalb einer zentralen MQTT-Serviceklasse realisiert. Diese Funktionen werden über die *MQTTnet*-Bibliothek implementiert. Nach erfolgreicher Verbindung zum Broker werden alle Topics durch die Wildcard „#“ abonniert. Bei einer

neuen Nachricht wird das bibliothekseigene Event `ApplicationMessageReceivedAsync` ausgelöst. In der MQTT-Serviceklasse registriert sich ein Eventhandler auf dieses Event. Der Eventhandler löst daraufhin ein weiteres Event aus, dem sowohl Topic und Payload als auch die Retain-Flag der eingehenden Nachricht übergeben wird.

Dieses zweistufige Eventmodell wurde genutzt, damit die verarbeitenden Klassen nicht zusätzlich die Informationen aus der Nachricht entnehmen müssen, sondern diese ohne zusätzliche wiederholte Aufbereitung der Nachrichteninformationen verwenden können.

Vor der Speicherung werden empfangene Payloads von den verarbeitenden Komponenten zunächst als JSON-Dokument geparkt. Dadurch stehen die Nachrichten in einer lesbaren Form zur Verfügung und können gleichzeitig als Grundlage für die Fehlererkennung, Prozessanalyse und Teilnehmerüberwachung verwendet werden.

Während der Implementierung zeigte sich, dass die im technischen Konzept vorgesehene strikt sequentielle Verarbeitung der Nachrichten nicht optimal für die Anforderungen der Benutzeroberfläche geeignet ist. Die Aufbereitung der Nachrichten erfolgt bereits unmittelbar nach dem Empfang, damit diese sowohl dargestellt als auch von den verschiedenen Analysefunktionen weiterverarbeitet werden können. Aus diesem Grund wurde die Verarbeitung in mehrere asynchron ausgeführte Teilschritte aufgeteilt.

Die fachliche Struktur der im Konzept definierten Verarbeitungsschritte blieb dabei unverändert. Die technische Umsetzung wurde jedoch hinsichtlich Performance und Benutzerfreundlichkeit angepasst, sodass mehrere Verarbeitungsschritte parallel ausgeführt werden können, ohne den Nachrichtenempfang oder die Benutzeroberfläche zu blockieren.

7.3 Persistente Speicherung der Kommunikationsdaten

Eine zentrale Anforderung der Anwendung bestand in der langfristigen Speicherung von MQTT-Nachrichten. Hierfür wurde eine SQLite-Datenbank eingesetzt. Neben den Kommunikationsdaten werden auch Fehler, Prozessabläufe, Nachrichtenvorlagen sowie Benutzereinstellungen in der Datenbank gespeichert. Dadurch können diese Informationen über mehrere Programmsitzungen hinweg erhalten bleiben und verbrauchen weniger Arbeitsspeicherplatz.

Die Speicherung der Daten in der Datenbank erfolgt durch SQL-Zugriffe in den Repository-

Klassen. Dabei greifen die Repository-Klassen zur Übersichtlichkeit jeweils auf lediglich eine Datentabelle in der Datenbank zu. Damit die Datenbank jedoch nicht zur gleichen Zeit beschrieben wird und so keine Fehler entstehen können, wurde ein sogenannter Semaphore für konkurrierende Schreibzugriffe verwendet. Ein Semaphore ist dabei ein Werkzeug, welches auf einzelne Prozesse warten kann, wenn Wartezeiten erwartet werden. Das wird hauptsächlich bei begrenzten Ressourcen wie Dateien verwendet, damit der Zugriff auf gemeinsame Ressourcen kontrolliert werden kann [6]. Für die Realisierung eines Semaphores wurde die SemaphoreSlim-Klasse herangezogen und bei jedem Schreibzugriff aufgerufen.

7.4 Realisierung des Paging-Konzepts

Bei der Implementierung des Paging-Konzepts wurde festgelegt, dass zu jedem Zeitpunkt nur ein begrenzter Ausschnitt der Nachrichten im Arbeitsspeicher gehalten wird. Diese Anzahl kann durch den Benutzer zwischen 20 und 60 Nachrichten konfiguriert werden. Kleinere Werte reduzieren den Arbeitsspeicherbedarf und beschleunigen die Aktualisierung der Benutzeroberfläche, während größere Werte einen besseren Überblick über den Nachrichtenverlauf ermöglichen. Die Parametrierbarkeit erlaubt somit eine Anpassung an verschiedene Voraussetzungen, wie beispielsweise die Leistungsfähigkeit des verwendeten Rechners oder die Anzahl der gleichzeitig verarbeiteten Nachrichten.

Für das Nachladen der Daten wurde zunächst das vollständige Neuladen der aktuellen Seite betrachtet. Dieses Vorgehen führte jedoch zu sichtbaren Sprüngen innerhalb der Darstellung und verursachte unnötige Datenbankzugriffe, wodurch die Performance der Anwendung beeinträchtigt wurde. Stattdessen wurde ein gleitende Paging-Konzept umgesetzt. Dabei werden beim Erreichen des oberen oder unteren Listendes lediglich zehn zusätzliche Nachrichten aus der Datenbank geladen. Gleichzeitig werden zehn nicht mehr benötigte Nachrichten aus dem Arbeitsspeicher entfernt. Dadurch wird die Performance der Anwendung verbessert und die Darstellung wirkt flüssiger. Außerdem bleibt die Anzahl der im Arbeitsspeicher befindlichen Objekte konstant.

Die Auswahl von zehn Nachrichten pro Nachladevorgang ergab sich aus praktischen Versuchen während der Entwicklung. Bei kleineren Werten traten aufgrund der häufigen Datenbankzugriffe vermehrt Nachladevorgänge auf. Größere Werte führten hingegen zu einer verzögerten Aktualisierung der Benutzeroberfläche und erhöhten den Speicherbedarf. Die gewählte Größe stellte einen geeigneten Kompromiss zwischen Reaktionsgeschwindigkeit und Ressourcenverbrauch dar.

Das Nachladen wird ereignisgesteuert ausgelöst. Sobald der Benutzer bis zum Anfang oder Ende der aktuell angezeigten Nachrichtenliste navigiert, wird ein entsprechendes Ereignis erzeugt. Die Anwendung bestimmt daraufhin den benötigten ID-Bereich und fordert ausschließlich die noch nicht geladenen Nachrichten aus der Datenbank an. Anschließend wird die bestehende `ObservableCollection` aktualisiert, ohne die komplette Ansicht neu aufzubauen.

Listing 7.1 zeigt den grundlegenden Aufbau der Paging-Funktionalität. Anhand der Identifikationsnummer des aktuell ersten beziehungsweise letzten Elements werden die benötigten Identifikationsnummern für die Nachrichten in der Datenbank ermittelt. Nach dem Laden der Datensätze werden die neuesten Elemente aus der Anzeige entfernt und die neuen Elemente an der entsprechenden Position eingefügt. Dadurch bleibt die Reihenfolge der Nachrichten in der Anzeige erhalten.

Listing 7.1: Programmcode für MQTT-Verbindung

```
1 public async Task<int> LoadNewerMessages()
2 {
3     if (messageCount <= MaxLoadedMessages)
4         return 0;
5     int startId = Messages[Messages.Count - 1].Id + 1;
6     int endId = Math.Min(messageCount, startId + loadnewMessages);
7     if (startId > messageCount)
8         return 0;
9     var messages = await _pagingService.LoadNewerMessages(startId, endId,
10 messageCount, loadnewMessages, SelectedTopic ?? "", SearchText ?? "");
11     Application.Current.Dispatcher.Invoke(() =>
12     {
13         for (int i = 0; i < messages.Count; i++)
14         {
15             Messages.RemoveAt(0);
16         }
17         for (int i = 0; i < messages.Count; i++)
18         {
19             Messages.Insert(Messages.Count, messages[i]);
20         }
21     });
22     return messages.Count;
23 }
```

Da neben vielen MQTT-Nachrichten auch viele Fehler und Prozessinformationen verarbeitet und angezeigt werden müssen, wurde das Paging-Konzept auch auf diese Daten angewendet. Dadurch

konnte die Performance der Anwendung weiter verbessert werden und der Arbeitsspeicherbedarf reduziert werden. Die Implementierung erfolgte analog zu der bereits beschriebenen Vorgehensweise für die MQTT-Nachrichten.

7.5 Teilnehmerüberwachung

Die Teilnehmerüberwachung übernimmt mehrere Aufgaben innerhalb der Anwendung. Zum einen werden automatisch Diagramme für jeden einzelnen Teilnehmer erstellt, in der OMAC-Zustände in Abhängigkeit von der Zeit dargestellt werden. Zum anderen werden Informationen wie die Zeit der letzten Nachricht sowie Position innerhalb der Verpackungslinie für andere Analysefunktionen bereitgestellt.

Die dafür benötigten Informationen werden in einer Serviceklasse bereitgestellt, die Teilnehmer zum einen durch das Auslesen der Linienkonfiguration erkennt, zum anderen aber auch durch die Analyse der Payloads OMAC-Zustände sowie Verbindungsstatusverläufe in der Datenbank speichert. Die Diagramme werden in einer anderen Serviceklasse durch die Übergabe der gespeicherten Zustände und Verläufe erstellt. Dazu dient die *OxyPlot*-Bibliothek, welche eine einfache Erstellung von Diagrammen ermöglicht. Die Diagramme werden anschließend in der Benutzeroberfläche angezeigt.

Die Teilnehmerüberwachung erfüllt damit nicht nur die Aufgabe der Verbindungsüberwachung, sondern stellt auch eine Grundlage für die Prozessanalyse bereit. Die erfassten Informationen können von den anderen Analysefunktionen genutzt werden, um Abläufe zu erkennen und Fehler zu identifizieren.

7.6 Realisierung der Automatisierungsablaufanalyse

Die Analyse von Automatisierungsabläufen stellt die zentrale Diagnosefunktion der Anwendung dar. Ziel der Implementierung war es, aus den empfangenen MQTT-Nachrichten verschiedene automatisierte Abläufe der Verpackungslinie zu erkennen und Fehler innerhalb dieser Abläufe zu identifizieren.

Aufgrund der unterschiedlichen Kommunikationsstrukturen und prozessspezifischen Abläufe verschiedener Verpackungslinien wurde der Umfang der automatischen Fehlererkennung bewusst auf eindeutig identifizierbare Ablaufverletzungen beschränkt. Fehler können nur dann erkannt werden, wenn neue Abläufe starten, während noch nicht alle erwarteten Nachrichten des vorherigen Ablaufs empfangen wurden. Die verschiedenen Anfangsnachrichten der Abläufe sowie die erwarteten Nachrichten können aus der Tabelle ?? entnommen werden.

Die Erkennung der erwarteten Nachrichten erfolgt durch die Ablauf-Serviceklasse. Diese erkennt aufgrund der definierten Anfangsnachrichten, dass ein neuer Ablauf gestartet wurde und speichert die dafür erwarteten Nachrichtenmodelle in jeweils für den Ablauf erstellten Listen. Die erwarteten Nachrichtenmodelle enthalten das erwartete MQTT-Topic sowie zusätzliche Zustandsinformationen, die innerhalb der Payload überprüft werden. In einer anderen Methode wird daraufhin jede Nachricht mit diesem Nachrichtenmodell verglichen und bei Übereinstimmung aus der Liste entfernt. Sobald die Liste der erwarteten Nachrichten leer ist, wird der Ablauf als abgeschlossen betrachtet und mit Endzeitstempel in der Datenbank abgespeichert. Sollte eine neue Anfangsnachricht empfangen werden, während noch nicht alle erwarteten Nachrichten des vorherigen Ablaufs empfangen wurden, wird der Ablauf als fehlerhaft betrachtet und ebenfalls mit Endzeitstempel in der Datenbank abgespeichert. Dazu wird die Fehlermeldung „Prozess nicht abgeschlossen“ in der Datenbank hinterlegt und an die Fehlererkennung weitergeleitet.

In der Benutzeroberfläche können sowohl aktive als auch bereits abgeschlossene Prozesse angezeigt werden. Neben Start- und Endzeitpunkt werden dabei auch die bereits empfangenen sowie die noch ausstehenden Nachrichten dargestellt. Dadurch kann der Benutzer nicht nur fehlerhafte Prozesse identifizieren, sondern auch nachvollziehen, an welcher Stelle eines Ablaufs die erwartete Kommunikation unterbrochen wurde.

7.7 Benutzeroberfläche

8 Testkonzept und Durchführung von Tests

8.1 Testkonzept

8.1.1 Testumgebung

Zur Validierung der Anforderungen an die entwickelte Überwachungs- und Diagnose-Applikation werden Funktions - und Performance-Tests durchgeführt. Ziel der Tests ist die Überprüfung, ob die in Kapitel 6 definierten Anforderungen vollständig umgesetzt wurden und unter realistischen Bedingungen zuverlässig funktionieren. Hierzu wird eine Verpackungslinie, die über alle Automatisierungsabläufe verfügt, als Testsystem verwendet. Dadurch kann die Anwendung unter realen Bedingungen getestet werden. Dazu werden folgende Funktionstests durchgeführt:

8.1.2 Funktionstests

Es wird ein korrekter Verbindungsaufbau und Verbindungsabbau sowie die Erkennung eines Verbindungsabbruchs getestet. Dazu werden teilweise falsche Broker-Ips und -Ports verwendet und die Verbindung zum Broker manuell getrennt. Zum bestehen des Tests muss der Verbindungsaufbau mit richtigen Informationen funktionieren und mit falschen Informationen abgelehnt werden. Des Weiteren muss ein Verbindungsabbau funktionieren und dadurch die Anwendung wieder auf Startzustand gebracht werden. Außerdem gilt der Test nur als erfolgreich, sollte ein Verbindungsabbruch innerhalb von drei Sekunden erkannt werden.

Des weiteren wird der Nachrichtenempfang, Paginierung und korrekte Filterung sowie Suche getestet. Dafür muss die Maschine mindestens eine halbe Stunde kontinuierlich Nachrichten schicken. Für einen erfolgreichen Test muss das Paging der Nachrichten problemlos von der ersten bis zur

letzten Nachricht in der Datenbank korrekt funktionieren, jeder angezeigte Topicfilter korrekte Nachrichten anzeigen und nach mindestens 20 oft gesuchten Schlagwörtern korrekt suchen können. Zu den Schlagwörtern gehören Maschinennamen, Zeiten und spezifische Nachrichtentypen. Durch diesen Test wird automatisch auch die funktionierende Speicherung in der SQLite-Datenbank überprüft, da diese Tests nur bei funktionierender Datenbank erfolgreich sind.

Um die Teilnehmererkennung zu testen, muss die Maschine die Retain-Nachrichten verschicken woraufhin manuell alle verschiedenen OMAC-Zustände für jeden Teilnehmer geschickt werden. Werden alle Teilnehmer der Linie erkannt und die OMAC-Zustände richtig aufgezeichnet ist dieser Test erfolgreich.

Die Ablauferkennung wird getestet, indem die Verpackungslinie die einzelnen Abläufe korrekt durchschreitet und die Anwendung diese auch korrekt abschließt. Daneben müssen manuell inkorrekte Abläufe versendet werden, die die Applikation mit einer Fehlermeldung anzeigt. Der Test ist erfolgreich, wenn die Abläufe im Ablaufübersicht-Reiter richtig angezeigt werden und mit der richtigen Fehlermeldung abgeschlossen werden.

Zuletzt wird die Fehlererkennung geprüft. Dazu müssen manuell Ablauf- und Strukturfehler gesendet werden. Dazu müssen alle Automatisierungsabläufe mit Fehlern simuliert werden und für jeden möglichen Strukturfehler einzelne Nachrichten erstellt werden. Erkennt die Applikation alle Fehler, ist auch dieser Test erfolgreich.

8.1.3 Performance-Tests

Damit die Anforderung an die Performance erfüllt wird, darf die Applikation während des ganzen Tests nicht abstürzen. Daneben soll kenntlich gemacht werden, wie leichtgewichtig die Applikation ist und wie viel Ressourcen sie verbraucht. Dazu werden über die Zeit der Tests kontinuierlich Arbeitsspeicherverbrauch und von der Anwendung verursachte Prozessorlast dokumentiert und ausgewertet. Der Performance-Test gilt als erfolgreich, wenn die Applikation während der gesamten Testdauer stabil betrieben werden kann und kein Absturz auftritt. Darüber hinaus muss die durchschnittliche CPU-Auslastung unter 10 Prozent liegen, während die maximale CPU-Auslastung 150 Prozent nicht überschreiten darf. Der durchschnittliche Arbeitsspeicherverbrauch muss unter 300 Megabyte liegen und der maximale Arbeitsspeicherverbrauch darf 450 Megabyte nicht überschreiten.

Zur Erfassung der Performance-Daten wurde ein selbst entwickeltes Python-Skript eingesetzt, das über die Bibliothek *psutil* in einem Intervall von 100 Millisekunden auf die Prozessdaten der Anwendung zugreift (siehe Listing B.6). Dadurch zeichnet das Skript CPU-Auslastung sowie Arbeitsspeichernutzung auf. Für die Ermittlung des Speicherverbrauchs wird der *Resident Set Size*-Wert verwendet. Dieser beschreibt den physischen Arbeitsspeicherplatz, der dem Prozess aktuell zugeordnet ist [25]. Im Gegensatz zu diesem Wert zeigt beispielsweise der Windows-Task-Manager den *Private-Working-Set*-Wert an. Dieser zeigt jedoch nur den aktuell im RAM befindlichen privaten Speicher des Prozesses an, während er reservierte oder ausgelagerte Speicherbereiche vernachlässigt [43]. Deswegen können diese Werte nicht miteinander verglichen werden. Das Skript speichert die aufgezeichnet Prozessdaten automatisch in eine *.csv*-Datei und erstellt ein Matlab-Datei, das aus den Prozessdaten automatisch die Graphen plottet.

8.2 Ergebnisse der Tests

Wie in Tabelle B.1 dargestellt wird, wurden 16 von 17 Funktionstests erfolgreich bestanden. Die Tests zum Verbindungsmanagement, Nachrichtenempfang, zur Teilnehmererkennung sowie zur Fehlererkennung verliefen erfolgreich.

Der Testfall zur vollständigen Ablaufferkennung konnte nicht vollständig abgeschlossen werden, da die für die Tests verwendete Testlinie nicht jeden von der Applikation unterstützten Automatisierungsablauf bereitstellte. Die vorhandenen und testbaren Abläufe wurden von der Anwendung erkannt und überwiegend korrekt erfolgreich oder fehlerhaft abgeschlossen. In einzelnen Fällen wurden Prozessabläufe fehlerhaft beendet, wenn ein neuer Ablauf erkannt wird, bevor die für die Auswertung benötigten OMAC-Zustände vollständig erfasst werden konnten.

Während der Durchführung der Funktionstests wurden zusätzlich fünf Performance-Messungen durchgeführt. Die Ergebnisse sind in Tabelle ?? dargestellt. Dabei wurden die durchschnittliche sowie die maximale CPU-Auslastung und der Arbeitsspeicherverbrauch der Anwendung erfasst. Eine grafische Auswertung der Messwerte zeigt (siehe Abbildung ??), dass sowohl die CPU-Auslastung als auch der Arbeitsspeicherverbrauch abhängig von der Anzahl der verarbeiteten MQTT-Nachrichten ansteigen und teilweise die vorher definierten Grenzwerte überschreiten. Während der gesamten Messdauer konnte die Anwendung jedoch ohne Abstürze betrieben werden.

9 Bewertung und Ergebnis

9.1 Bewertung der Testergebnisse

Die durchgeführten Funktionstests zeigen, dass die wesentlichen Anforderungen an die entwickelte Applikation erfüllt wurden. Die nicht vollständig erfolgreiche Validierung der vollständigen Ablauferkennung ist auf Einschränkung der Testumgebung und auf das Verbinden während des Betriebs zurückzuführen. Durch eine fehlende Unterstützung aller Automatisierungsabläufe der Testlinie, konnten nicht alle in der Applikation implementierte Abläufe getestet werden. Die fehlerhafte Verarbeitung der Prozessabläufe, die als erstes erkannt werden, kann darauf zurückgeführt werden, dass die Applikation eine Grundlage an Informationen für korrekte Erkennung von Prozessen benötigt. Diese Informationen werden jedoch erst im Verlauf der aktiven Verbindung über MQTT geschickt, wodurch Abläufe dann falsch erkannt werden. Dieses Fehlverhalten der Anwendung könnte durch Hinzufügen von Bedingungen behoben werden, wodurch Abläufe erst erkannt werden sollen, wenn genügend Informationen bereitgestellt sind. Da sich aber die sofortige Aufzeichnung von Automatisierungsabläufen als wichtiger eingestuft wird, als jeden einzelnen Prozess mit der richtigen Information abzuschließen, wird diese Einschränkung als tolerierbar bewertet.

Die definierten Performance-Ziele konnten nicht vollständig erreicht werden. Insbesondere die CPU-Auslastung überschritt die festgelegten Grenzwerte während mehrerer Testläufe. Ursache hierfür war die bewusst hohe und kontinuierliche Nachrichtenlast während der Messungen. Die Anwendung blieb trotz erhöhter Prozessor-Auslastung jederzeit stabil und konnte alle Nachrichten ohne Datenverluste verarbeiten. Aus diesem Grund kann die Performance trotz fehlgeschlagener Tests als ausreichend bewertet werden.

9.2 Umsetzung der Anforderungen

9.2.1 Nachrichtendarstellung und Filterung

Die Anforderung zur Anzeige, Speicherung und Filterung von MQTT-Nachrichten wurde vollständig umgesetzt. Die Applikation ermöglicht sowohl die Filterung nach Topics als auch die Volltextsuche innerhalb der gespeicherten Nachrichten. Darüber hinaus können drei verschiedene Anzeigemöglichkeiten der Nachrichten ausgewählt werden, wodurch die Anwendung personalisierbar wird.

9.2.2 Fehlererkennung

Die automatische Erkennung von Strukturfehlern sowie verschiedener Prozessfehler wurde erfolgreich implementiert und konnte durch die Funktionstests validiert werden. Neben der reinen Anzeige von Fehlern können diese nach Ablauf- beziehungsweise Strukturfehler gefiltert werden. Außerdem wurde das Paging-Konzept auch für die Fehler umgesetzt, wodurch diese ressourcenarm angezeigt werden können.

9.2.3 Teilnehmerüberwachung

Die Teilnehmerüberwachung erkennt automatisch neue Teilnehmer und visualisiert deren Zustände sowie deren Verbindungsstatus, wodurch die Anforderungen dafür erfüllt wurden.

9.2.4 Automatisierungsablauf-Erkennung

Bei der Ablauferkennung bildeten sich bei der Durchführung der Funktionstests geringfügige Fehler heraus. Trotz dieser Einschränkungen bietet die implementierte Ablauferkennung einen deutlichen Mehrwert für die Analyse von Produktionsabläufen. Prozessereignisse können automatisch erkannt und fehlerhafte Abläufe durch entsprechende Fehlermeldungen hervorgehoben werden. Dadurch wird die Fehlersuche gegenüber einer rein manuellen Analyse erheblich vereinfacht.

9.2.5 Performance

Durch die Nutzung asynchroner Verarbeitung sowie eines Paging-Konzeptes konnte eine stabile Verarbeitung großer Nachrichtenmengen erreicht werden. Die definierten Zielwerte für CPU- und Arbeitsspeicherverbrauch wurden jedoch nicht in allen Testfällen eingehalten.

9.2.6 Erweiterbarkeit und Zukunftstauglichkeit

Durch die Verwendung der MVVM-Architektur sowie der Trennung von Services, Repositories und Benutzeroberfläche wurde eine Grundlage für zukünftige Erweiterungen geschaffen. Für den Fall eines zukünftigen Umstiegs auf andere Betriebssysteme ist die Applikation nicht sofort übertragbar und einsetzbar. Dafür müsste die Anwendung auf ein anderes Framework übertragen werden, was jedoch durch die MVVM-Architektur stark vereinfacht wird.

Die Anforderung an konfigurierbare Prüfmechanismen erfüllt der MQTT Analyzer nicht. Dafür ist der Prozessablauf einer jeden Maschine zu modular aufgebaut, wodurch die Umsetzung dieser Anforderung für den Umfang dieser Projektarbeit einen zu großen Aufwand mit sich bringen würde.

Insgesamt zeigt die Bewertung, dass die entwickelte Anwendung für den Einsatz im Produktionsumfeld geeignet ist. Durch die Kombination aus Nachrichtenanalyse, automatischer Fehlererkennung, Teilnehmerüberwachung und Ablaufferkennung bietet sie einen deutlichen Mehrwert gegenüber den bisher eingesetzten MQTT-Werkzeugen und unterstützt. Der Anwender wird dadurch bei der Inbetriebnahme, Analyse und Fehlerdiagnose von Verpackungslinien wirksam unterstützt.

10 Fazit und Ausblick

10.1 Überblick über das Projekt

Ziel dieser Arbeit war die Entwicklung einer Überwachungs- und Diagnose-Applikation zur Analyse der MQTT-basierten Maschinenkommunikation in Verpackungslinien. Hierzu wurde zunächst die bestehende Situation untersucht und verschiedene vorhandene Werkzeuge hinsichtlich ihrer Eignung bewertet. Auf Basis der identifizierten Anforderungen wurde anschließend eine eigenständige Anwendung entwickelt.

Die entwickelte Applikation ermöglicht die Speicherung und Analyse von MQTT-Nachrichten, die Überwachung von Kommunikationsteilnehmern sowie die automatische Erkennung von Struktur- und Prozessfehlern. Darüber hinaus können verschiedene Automatisierungsabläufe erkannt und visualisiert werden.

Die durchgeführten Funktionstests zeigen, dass die wesentlichen Anforderungen erfolgreich umgesetzt werden konnten. Trotz einzelner Einschränkungen bei der vollständigen Validierung aller Automatisierungsabläufe und der nicht vollständig erreichten Performance-Ziele konnte die Anwendung stabil betrieben werden.

Besonders hervorzuheben ist, dass die entwickelte Software bereits von Softwareentwicklern im Arbeitsumfeld genutzt wird. Dadurch konnte gezeigt werden, dass die Lösung nicht nur prototypischen Charakter besitzt, sondern einen praktischen Nutzen für die Inbetriebnahme und Fehlersuche an Verpackungslinien bietet.

10.2 Zukünftige Entwicklung der Anwendung

10.2.1 Weitere Automatisierungsabläufe

Die aktuelle Implementierung unterstützt bereits die wichtigsten Automatisierungsabläufe. Für eine umfassendere automatische Analyse der Verpackungslinien durch die Software können weitere Ablaufvarianten ergänzt werden. Dafür muss lediglich die *ProcessService*-Datei erweitert werden.

10.2.2 Verbesserte Konfigurierbarkeit

Darüber hinaus kann die fehlende Anforderung der konfigurierbaren Analyse- und Validierungslogik umgesetzt werden. Dadurch sollen neue Prüfmechanismen ohne Änderungen am Quellcode ergänzt werden können.

10.2.3 Linux / HMI

Ein weiterer Entwicklungsschwerpunkt ergibt sich aus der geplanten Umstellung zukünftiger Bedienoberflächen auf Linux-basierte Systeme. Da die vorliegende Anwendung auf WPF basiert, ist diese aktuell an Windows gebunden. Für einen zukünftigen Einsatz direkt auf den HMIs könnte die Anwendung auf ein plattformübergreifendes Framework wie .NET MAUI oder Avalonia portiert werden. Die verwendete MVVM-Architektur erleichtert eine solche Migration, da große Teile der Anwendungslogik bereits von der Benutzeroberfläche getrennt sind. Dadurch könnte die entwickelte Lösung zukünftig direkt auf den Bediengeräten der Maschine eingesetzt werden und den Anwendern ohne zusätzliche Hardware zur Verfügung stehen.

Quellenverzeichnis

Gedruckte Quellen

- [4] Wolfgang Babel. *Systemintegration in Industrie 4.0 Und IoT*. 1. Aufl. Wiesbaden: Springer Vieweg. ISBN: 978-3-658-42987-4.
- [5] Abdullrahman Yousif Bahar u. a. „Survey on Features and Comparisons of Programming Languages (PYTHON, JAVA, AND C#)“. In: *2022 ASU International Conference in Emerging Technologies for Sustainability and Intelligent Systems (ICETISIS)*. 2022 ASU International Conference in Emerging Technologies for Sustainability and Intelligent Systems (ICETISIS). Juni 2022, S. 154–163. DOI: 10 . 1109 / ICETISIS55481 . 2022 . 9888839. URL: <https://ieeexplore.ieee.org/abstract/document/9888839> (besucht am 31.07.2026).
- [9] Frederic Desbiens. *Building Enterprise IoT Solutions with Eclipse IoT Technologies : An Open Source Approach to Edge Computing*. 1. Aufl. Berkeley: Apress, 2023. ISBN: 978-1-4842-8882-5.
- [12] HIVEMQ, Hrsg. *MQTT & MQTT 5 Essentials*. Landshut. ISBN: 978-3-00-067913-1.
- [14] Jörg Kastning. „TLS/SSL Kochbuch“. 9. Aug. 2016.
- [16] Jay Kreibich. *Using SQLite*. Ö'Reilly Media, Inc.", 17. Aug. 2010. 526 S. ISBN: 978-0-596-52118-9. Google Books: HFIM47wp0X0C.
- [20] Pieter Nijs. *The MVVM Pattern in .NET MAUI: The Definitive Guide to Essential Patterns, Best Practices, and Techniques for Cross-Platform App Development*. Packt Publishing Ltd, 30. Nov. 2023. 386 S. ISBN: 978-1-80512-070-4. Google Books: 7HTkEAAAQBAJ.
- [21] OASIS. *MQTT Version 5.0*. 7. März 2019.
- [22] Beate Ottenwalder. „Churn in dynamischen Publish/Subscribe Systemen“. Projektarbeit. Stuttgart: Universität Stuttgart, 11. Dez. 2007.
- [23] Robert Panther. *Datenbankentwicklung Lernen Mit SQL Server 2017: Der Praxisorientierte Grundkurs*. 2. Aufl. Heidelberg: O'Reilly, 2019. XII, 495. ISBN: 978-3-96010-222-9.

- [24] Valentin Plenk. *Angewandte Netzwerktechnik Kompakt*. 3. Aufl. Oberkotzau: Springer Vieweg, 22. Juni 2024. 365 S. ISBN: 978-3-658-43529-5. URL: https://link.springer.com/chapter/10.1007/978-3-658-43530-1_7.
- [27] Vaskaran Sarcar. *Parallel Programming with C# and .NET*. 1. Aufl. Barkeley: Apress, 2024. ISBN: 979-8-8688-0488-5.
- [28] Nico Schmidt. *Vergleich Und Bewertung von Kommunikationskonzepten Für Microservices imBereichdes Internetof Things*. Leipzig, 17. Mai 2021.
- [29] Matthias Schubert. *Datenbanken*. 2. Aufl. Wiesbaden: B. G. Teubner Verlag, Apr. 2007. 351 S. ISBN: 978-3-8351-0163-0.
- [35] Andrew W. Troelsen und Philip Japikse. *Pro C# with .NET 6: Foundational Principals and Practices in Programming*. 11. New York: Apress, 2022. ISBN: 978-1-4842-7869-7.
- [37] Lukas Vileikis. *Hacking MySQL*. Siauliai: Apress Berkeley, 2024. ISBN: 979-8-8688-0979-8.
- [39] Jörg Wegener. *WPF 4.5 und XAML: Grafische Benutzeroberflächen für Windows inkl. Entwicklung von Windows Store Apps*. Carl Hanser Verlag GmbH Co KG, 6. Dez. 2012. 706 S. ISBN: 978-3-446-43541-4. Google Books: Z7NPAGAAQBAJ.
- [40] Joachim Wietzke. *Embedded Technologies*. Xpert.Press. Karlsruhe: Springer Vieweg, Jan. 2012. ISBN: 978-3-642-23995-3.

Online-Quellen

- [1] adegeo. *XAML Language Overview - WPF*. URL: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/xaml/> (besucht am 24.07.2026).
- [2] ajcvickers. *Overview - Microsoft.Data.Sqlite*. URL: <https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/> (besucht am 31.07.2026).
- [3] *Asyncio — Asynchronous I/O*. Python documentation. URL: <https://docs.python.org/3/library/asyncio.html> (besucht am 23.07.2026).
- [6] BillWagner. *Semaphor und SemaphoreSlim - .NET*. URL: <https://learn.microsoft.com/de-de/dotnet/standard/threading/semaphore-and-semaphoreslim> (besucht am 22.06.2026).
- [7] chkr1011. *mqttMultimeter*. Github Enterprise. 7. Feb. 2026. URL: <https://github.com/chkr1011/mqttMultimeter> (besucht am 08.05.2026).
- [8] *Connect with MQTT.Fx | EMQX Cloud Docs*. URL: https://docs.emqx.com/en/cloud/latest/connect_to_deployments/mqttx.html (besucht am 27.04.2026).

- [10] Jens Deters. *MQTT Testing and Debugging Using MQTT.fx HiveMQ Cloud Edition*. URL: <https://www.hivemq.com/blog/mqtt-testing-debugging-using-mqttfx-hivemq-cloud/> (besucht am 24.04.2026).
- [11] dotnet-bot. *SqlConnection Class (Microsoft.Data.SqlClient)*. URL: <https://learn.microsoft.com/en-us/dotnet/api/microsoft.data.sqlclient.sqlconnection?view=sqlclient-dotnet-core-6.1> (besucht am 31.07.2026).
- [13] *IPCOMM, Protocols: MQTT*. URL: <https://www.ipcomm.de/protocol/MQTT/en/sheet.html> (besucht am 24.07.2026).
- [15] Stefan Kirsch. *XAML Mit WPF, UWP, WinUI 3 Und .NET MAUI - Kirsch Software*. KIRSCH Software. 20. Juni 2024. URL: <https://kirsch-software.de/blog/XAML-Frameworks-WPF-UWP-WinUI3-MAUI> (besucht am 06.05.2026).
- [17] *Model-View-ViewModel - .NET*. URL: <https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm> (besucht am 23.06.2026).
- [18] *MQTT - Message Queue Telemetry Transport*. URL: <https://www.elektronik-kompdi-um.de/sites/net/2204051.htm> (besucht am 24.04.2026).
- [19] *MQTT.fx: Grafisches Werkzeug zum MQTT-Debugging*. codecentric AG. URL: <https://www.codecentric.de/wissens-hub/blog/mqtt-fx-grafisches-werkzeug-zum-mqtt-debugging> (besucht am 24.04.2026).
- [25] *Psutil Documentation — Psutil 7.2.2 Documentation*. URL: <https://psutil.readthedocs.io/stable/> (besucht am 29.07.2026).
- [26] *Rider: The Cross-Platform .NET and Game Development IDE from JetBrains*. JetBrains. URL: <https://www.jetbrains.com/rider/> (besucht am 31.07.2026).
- [30] Josh Smith. *Patterns - WPF Apps With The Model-View-ViewModel Design Pattern*. Feb. 2009. URL: <https://learn.microsoft.com/en-us/archive/msdn-magazine/2009/february/patterns-wpf-apps-with-the-model-view-viewmodel-design-pattern> (besucht am 23.06.2026).
- [31] EMQX Cloud Team. *How to Use MQTT in C# with MQTTnet*. www.emqx.com. URL: <https://www.emqx.com/en/blog/connecting-to-serverless-mqtt-broker-with-mqttnet-in-csharp> (besucht am 28.04.2026).
- [32] ZTABS Team. *Rebuild vs. Refactor Legacy Software: Decision Guide*. Sun Feb 22 2026 00:00:00 GMT+0000 (Coordinated Universal Time). URL: <https://ztabs.co/blog/when-to-rebuild-vs-refactor-legacy-software> (besucht am 11.06.2026).

- [33] Harshit Tiwari. *Virtual Memory Explained: Paging, Segmentation, and Handling Page Faults - NamasteDev Blogs*. 23. Okt. 2025. URL: <https://namastedev.com/blog/virtual-memory-explained-paging-segmentation-and-handling-page-faults/> (besucht am 02.06.2026).
- [34] *Tkinter — Python Interface to Tcl/Tk*. Python documentation. URL: <https://docs.python.org/3/library/tkinter.html> (besucht am 23.07.2026).
- [36] *Übersicht über .NET Framework - .NET Framework*. URL: <https://learn.microsoft.com/de-de/dotnet/framework/get-started/overview> (besucht am 15.06.2026).
- [38] *Was ist Visual Studio?* URL: <https://learn.microsoft.com/de-de/visualstudio/get-started/visual-studio-ide?view=visualstudio> (besucht am 06.05.2026).
- [41] *Working with C#*. URL: <https://code.visualstudio.com/docs/languages/csharp> (besucht am 06.05.2026).
- [42] Gavin Wright. *Definition Agnostisch*. ComputerWeekly.de. URL: <https://www.computerweekly.com/de/definition/Agnostisch> (besucht am 23.04.2026).
- [43] Pavel Yosifovich. *Memory Information in Task Manager*. Pavel Yosifovich. 12. Apr. 2023. URL: <https://scorpiosoftware.net/2023/04/12/memory-information-in-task-manager/> (besucht am 29.07.2026).

Abbildungsverzeichnis

4.1	Subscribe-Reiter in MQTT.fx	15
4.2	MQTT Explorer	17
4.3	Anzeigen der Nachrichten in einer Baumstruktur im MQTMMultimeter	18
4.4	Nachrichtenverlauf im MQTMMultimeter	19
6.1	Grundlegendes Konzept für die GUI	42
B.1	Floatchart-Diagramm für die Nachrichtenverarbeitung	65

Tabellenverzeichnis

A.1	Liste der verwendeten Künstliche Intelligenz basierten Werkzeuge	62
B.1	Ergebnisse der durchgeführten Funktionstests	73

A Anmerkung zur Nutzung von Künstlicher Intelligenz

Im Rahmen dieser Arbeit wurden Künstliche Intelligenz (KI) basierte Werkzeuge benutzt. Tabelle A.1 gibt eine Übersicht über die verwendeten Werkzeuge und den jeweiligen Einsatzzweck.

Tabelle A.1: Liste der verwendeten KI-basierten Werkzeuge

Werkzeug	Beschreibung der Nutzung
Microsoft Copilot	• Unterstützung bei der Programmierung (z. B. Code-Vorschläge und Strukturierung) • Hilfe bei der sprachlichen Formulierung und Verbesserung von Satzstrukturen (gesamt) • Zusammenfassung und Aufbereitung von wissenschaftlichen Texten als Grundlage für eigene Formulierungen (gesamt)
DeepL	• Verbesserung von Satzstrukturen und Formulierungen (gesamt)

B Ergänzungen

B.1 Details zu bestimmten theoretischen Grundlagen

B.2 Weitere Details, welche im Hauptteil den Lesefluss behindern

Listing B.1: Programmcode für Verarbeitung und Darstellung empfangener MQTT-Nachrichten

```
1 public InflightPageViewModel(MqttClientService mqttClientService)
2 {
3     _mqttClientService = mqttClientService;
4     // Registrierung auf eingehende MQTT-Nachrichten
5     _mqttClientService.ApplicationMessageReceived += OnApplicationMessageReceived;
6     // Reaktiver Filter basierend auf Benutzereingabe
7     var filter = this.WhenAnyValue(x => x.FilterText)
8         .Throttle(TimeSpan.FromMilliseconds(800))
9         .Select(BuildFilter);
10    // Datenpipeline: Quelle -> Filter -> UI-Bindung
11    _itemsSource.Connect()
12        .Filter(filter)
13        .ObserveOn(RxApp.MainThreadScheduler)
14        .Bind(out _items)
15        .Subscribe();
16 }
17 // Verarbeitung eingehender Nachrichten
18 Task OnApplicationMessageReceived(MqttApplicationMessageReceivedEventArgs
    eventArgs)
19 {
20     return AppendMessage(eventArgs.ApplicationMessage);
21 }
22 // Hinzufügen neuer Nachrichten zur Datenquelle
23 public Task AppendMessage(MqttApplicationMessage message)
24 {
```

```

25     return Dispatcher.UIThread.InvokeAsync(() =>
26     {
27         var newItem = CreateItemViewModel(message);
28         _itemsSource.Add(newItem);
29     }).GetTask();
30 }
31 // Erstellung des Filters
32 Func<InflightPageItemViewModel, bool> BuildFilter(string? searchText)
33 {
34     if (string.IsNullOrEmpty(searchText))
35     {
36         return t => true;
37     }
38     return t => t.Topic.Contains(searchText, StringComparison.OrdinalIgnoreCase);
39 }

```

Listing B.2: Speichern von Templates in Datenbank

```

1 public async Task<bool> AddTemplate(PublishModel model)
2 {
3     await semaphore.WaitAsync();
4     try
5     {
6         using var connection = new SqliteConnection(_connectionString);
7
8         await connection.OpenAsync();
9         // Prüfen ob Name schon existiert
10        var checkCommand = connection.CreateCommand();
11
12        checkCommand.CommandText =
13        @"
14            SELECT COUNT(*)
15            FROM PublishTemplates
16            WHERE Name = $name
17        ";
18
19        checkCommand.Parameters.AddWithValue("$name", model.Name);
20
21        var count = (long)await checkCommand.ExecuteScalarAsync();
22
23        // Falls Name schon existiert -> nichts speichern
24        if (count > 0)
25            return false;
26
27        // Speichern

```

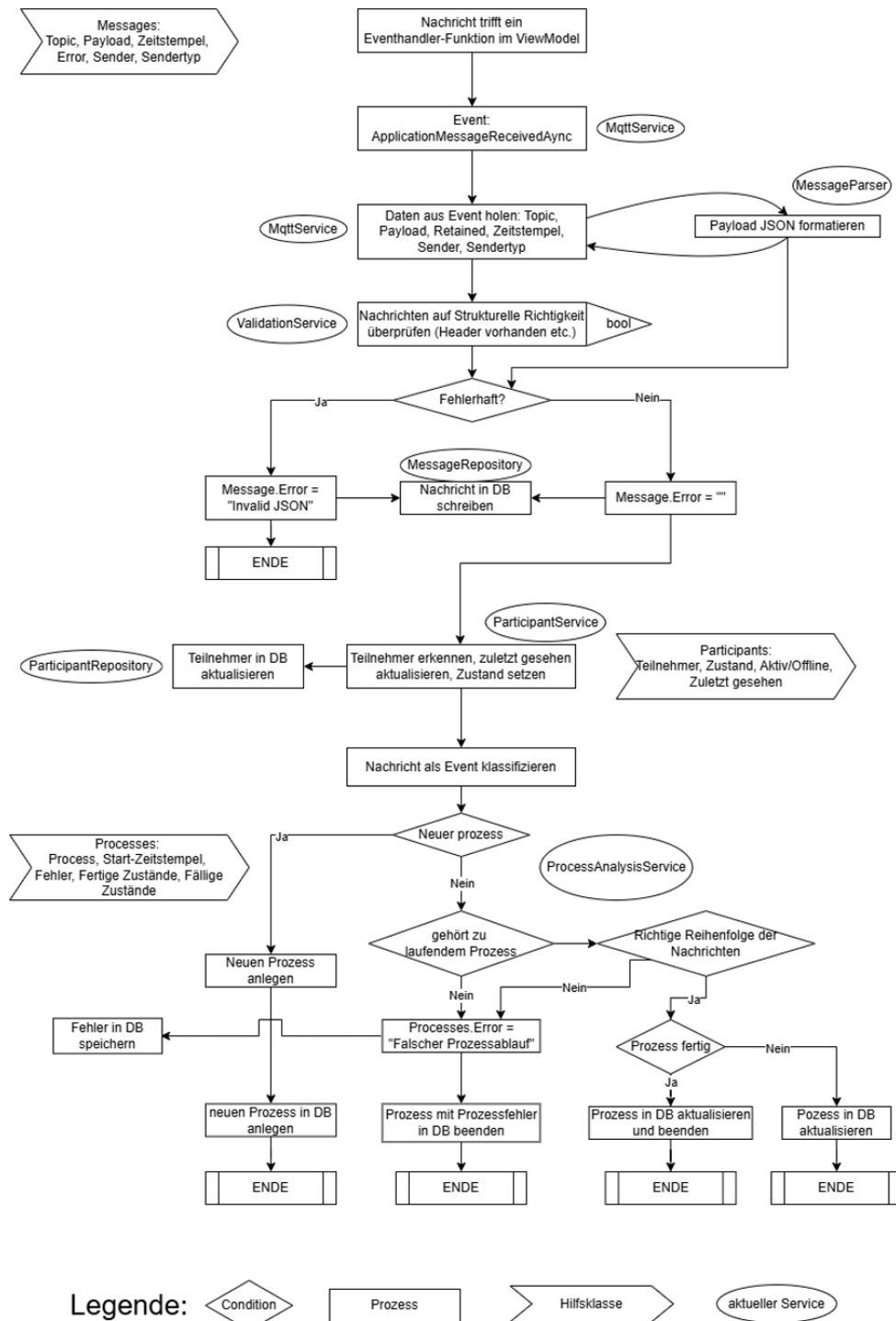


Abbildung B.1: Floatchart-Diagramm für die Nachrichtenverarbeitung

```
28     var command = connection.CreateCommand();
29
30     command.CommandText =
31     @"
32         INSERT INTO PublishTemplates
33         (Name, Topic, Header, Payload)
34
35         VALUES
36         ($name, $topic, $header, $payload);
37     ";
38
39     command.Parameters.AddWithValue("$name", model.Name);
40     command.Parameters.AddWithValue("$topic", model.Topic);
41     command.Parameters.AddWithValue("$header", model.Header);
42     command.Parameters.AddWithValue("$payload", model.Payload);
43
44     await command.ExecuteNonQueryAsync();
45 }
46 finally
47 {
48     semaphore.Release();
49 }
50 return true;
51 }
```

Listing B.3: Automatische Anordnung der Publish-Buttons durch ein ItemsControl-Feld

```
1 <ItemsControl Grid.Column="0" ItemsSource="{Binding CustomButtons}">
2     <ItemsControl.ItemsPanel>
3         <ItemsPanelTemplate>
4             <WrapPanel/>
5         </ItemsPanelTemplate>
6     </ItemsControl.ItemsPanel>
7     <ItemsControl.ItemTemplate>
8         <DataTemplate>
9             <Border Margin="5">
10                 <Grid>
11                     <Button Content="{Binding Name}"
12                         Background="{Binding Color}"
13                         Foreground="White"
14                         Width="200"
15                         Height="40"
16
17                         Command="{Binding DataContext.PublishButtonCommand,
18                             RelativeSource={RelativeSource
```

```

19         AncestorType=UserControl}}"
20         CommandParameter="{Binding}"
21     />
22     <Button Content="X"
23         Width="20"
24         Height="20"
25         HorizontalAlignment="Right"
26         VerticalAlignment="Top"
27         Command="{Binding DataContext.DeleteButtonCommand,
28             RelativeSource={RelativeSource
29                 AncestorType=UserControl}}"
30         CommandParameter="{Binding}" />
31 </Grid>
32 </Border>
33 </DataTemplate>
34 </ItemsControl.ItemTemplate>
35 </ItemsControl>

```

Listing B.4: Baumstruktur in XAML

```

1 <TreeView x:Name="TopicTree"
2     ItemsSource="{Binding FilteredTopics}"
3     SelectedItemChanged="TopicTree_SelectedItemChanged"
4     BorderThickness="0"
5     Background="Transparent">
6
7     <TreeView.Resources>
8
9         <HierarchicalDataTemplate
10             DataType="{x:Type model:TopicTreeNode}"
11             ItemsSource="{Binding Children}">
12
13             <TextBlock Text="{Binding DisplayName}"
14                 FontSize="14"
15                 Margin="2" />
16
17         </HierarchicalDataTemplate>
18
19     </TreeView.Resources>
20     <TreeView.ItemContainerStyle>
21         <Style TargetType="TreeViewItem">
22
23             <Setter Property="IsExpanded"
24                 Value="{Binding IsExpanded, Mode=TwoWay}" />

```

```

25
26         <Setter Property="IsSelected"
27 Value="{Binding IsSelected, Mode=TwoWay}" />
28
29     </Style>
30 </TreeView.ItemContainerStyle>
31
32 </TreeView>

```

Listing B.5: Speicherung der gesamten Topic-Hierarchie in TopicTreeNode

```

1 public async Task<bool> AddTemplate(PublishModel model)
2 {
3     await semaphore.WaitAsync();
4     try
5     {
6         using var connection = new SqliteConnection(_connectionString);
7
8         await connection.OpenAsync();
9         // Prüfen ob Name schon existiert
10        var checkCommand = connection.CreateCommand();
11
12        checkCommand.CommandText =
13        @"
14            SELECT COUNT(*)
15            FROM PublishTemplates
16            WHERE Name = $name
17        ";
18
19        checkCommand.Parameters.AddWithValue("$name", model.Name);
20
21        var count = (long)await checkCommand.ExecuteScalarAsync();
22
23        // Falls Name schon existiert -> nichts speichern
24        if (count > 0)
25            return false;
26
27        // Speichern
28        var command = connection.CreateCommand();
29
30        command.CommandText =
31        @"
32            INSERT INTO PublishTemplates
33            (Name, Topic, Header, Payload)
34

```

```

35         VALUES
36         ($name, $topic, $header, $payload);
37     ";
38
39     command.Parameters.AddWithValue("$name", model.Name);
40     command.Parameters.AddWithValue("$topic", model.Topic);
41     command.Parameters.AddWithValue("$header", model.Header);
42     command.Parameters.AddWithValue("$payload", model.Payload);
43
44     await command.ExecuteNonQueryAsync();
45 }
46 finally
47 {
48     semaphore.Release();
49 }
50 return true;
51 }

```

Listing B.6: Aufzeichnung von CPU-Auslastung und Arbeitsspeicherverbrauch

```

1 import time
2 import csv
3 import sys
4 import psutil
5
6 TARGET_PROCESS = "MQTTAnalyzer.exe"
7 INTERVAL_SECONDS = 0.1 # 100 ms
8 CSV_FILENAME = "mqtt_analyzer_log.csv"
9 MATLAB_SCRIPT = "plot_mqtt_data.m"
10
11
12 def find_process(process_name):
13     for proc in psutil.process_iter(['pid', 'name']):
14         try:
15             if proc.info['name'] and proc.info['name'].lower() == process_name.
lower():
16                 return proc
17         except (psutil.NoSuchProcess,
18                 psutil.AccessDenied,
19                 psutil.ZombieProcess):
20             pass
21
22     return None
23
24

```



```
25
26 def main():
27     print(f"Suche nach Prozess {TARGET_PROCESS} ...")
28
29     proc = find_process(TARGET_PROCESS)
30
31     if proc is None:
32         print(f"Fehler: Prozess '{TARGET_PROCESS}' wurde nicht gefunden.")
33         sys.exit(1)
34
35     print(f"Prozess gefunden! (PID: {proc.pid})")
36     print(f"Messung alle {INTERVAL_SECONDS} Sekunden")
37     print("Beenden mit Strg+C\n")
38
39     proc.cpu_percent()
40     time.sleep(INTERVAL_SECONDS)
41
42     max_cpu = 0.0
43     max_ram_mb = 0.0
44     total_cpu = 0.0
45     total_ram = 0.0
46     count = 0
47
48     start_time = time.time()
49
50     with open(CSV_FILENAME, "w", newline="", encoding="utf-8") as file:
51         writer = csv.writer(file)
52
53         writer.writerow([
54             "Zeitstempel",
55             "Zeit_Sekunden",
56             "CPU_Prozent",
57             "RAM_MB"
58         ])
59
60     try:
61         while True:
62             if not proc.is_running():
63                 print("\nProzess wurde beendet.")
64                 break
65
66             elapsed = time.time() - start_time
67
68             timestamp = (
```

```

69         time.strftime("%Y-%m-%d %H:%M:%S")
70         + f".{int((elapsed % 1) * 1000):03d}"
71     )
72
73     cpu = proc.cpu_percent(interval=None)
74     ram_mb = proc.memory_info().wset / (1024 * 1024)
75
76     max_cpu = max(max_cpu, cpu)
77     max_ram_mb = max(max_ram_mb, ram_mb)
78
79     total_cpu += cpu
80     total_ram += ram_mb
81     count += 1
82
83     writer.writerow([
84         timestamp,
85         f"{elapsed:.3f}",
86         f"{cpu:.2f}",
87         f"{ram_mb:.2f}"
88     ])
89
90     file.flush()
91
92     time.sleep(INTERVAL_SECONDS)
93
94     except KeyboardInterrupt:
95         print("\nAufzeichnung manuell beendet.")
96
97     if count > 0:
98         avg_cpu = total_cpu / count
99         avg_ram = total_ram / count
100
101         print("\n" + "=" * 50)
102         print(f"Zusammenfassung ({count} Messungen)")
103         print("=" * 50)
104         print(f"Max CPU-Auslastung: {max_cpu:.2f} %")
105         print(f"Durchschnittliche CPU-Auslastung: {avg_cpu:.2f} %")
106         print(f"Maximaler RAM-Verbrauch: {max_ram_mb:.2f} MB")
107         print(f"Durchschnittlicher RAM-Verbrauch: {avg_ram:.2f} MB")
108
109     print(f"\nMATLAB-Skript erstellt: {MATLAB_SCRIPT}")
110     print(f"CSV-Datei erstellt: {CSV_FILENAME}")
111
112

```

```

113 if __name__ == "__main__":
114     main()

```

Test-ID	Testfall	Testdurchführung	Erwartetes Ergebnis	Ergebnis
FT-01	Verbindungs-Aufbau	Verbindung mit gültigem Broker herstellen	Verbindung wird aufgebaut und angezeigt	X
FT-02	Ungültige Broker-IP	Falsche IP-Adresse eintragen	5 Sekunden lang versuchen, dann Fehlermeldung anzeigen	X
FT-03	Ungültiger Broker-Port	Falschen Port eintragen	5 Sekunden lang versuchen, dann Fehlermeldung anzeigen	X
FT-04	Manuelle Verbindungstrennung	Brokerverbindung manuell trennen	Verbindung wird getrennt und angezeigt	X
FT-05	Verbindungs-Abbruch	Netzwerkkabel wird entfernt	Verbindungsabbruch wird in unter 3 Sekunden erkannt und Fehlermeldung wird angezeigt	X
FT-06	Nachrichtenempfang	Maschine sendet kontinuierlich Nachrichten; empfangene Nachrichten werden mit MQTT.fx verglichen	Alle Nachrichten werden empfangen	X
FT-07	Pagination	Nachrichtenliste mit mindestens 3000 Nachrichten wird ganz durchgeblättert	Alle Nachrichten werden korrekt nachgeladen und gelöscht	X
FT-08	Topic-Filterung	Verschiedene Topic-Filter wählen	Alle passenden Nachrichten werden gefunden und angezeigt	X

FT-09	Suchfunktion	Verschiedene Teilnehmernamen, Uhrzeiten und Nachrichten werden eingegeben	Alle passenden Nachrichten werden gefunden und angezeigt	X
FT-10	Datenbank-Speicherung	Nachrichten empfangen und Anwendung erneut laden	Gespeicherte Nachrichten bleiben erhalten	X
FT-11	Teilnehmer-Erkennung	Retain-Nachrichten empfangen	Alle Teilnehmer ohne „integrated By“ richtig anzeigen	X
FT-12	Zustands- und Statuserfassung	Alle OMAC-States von jedem Teilnehmer senden und empfangen	Zustände werden korrekt gespeichert und angezeigt	X
FT-13	Regulärer Prozessablauf	Verpackungslinie durchläuft Prozess und beendet ihn korrekt	Prozess wird erkannt und ohne Fehler abgespeichert	–
FT-14	Fehlerhafter Prozess	Maschine durchläuft Prozess, beendet diesen jedoch nicht	Prozess wird erkannt und mit Fehler abgespeichert	X
FT-15	Prozessfehler erkennen	Maschine durchläuft Prozess, beendet diesen jedoch nicht	Prozessfehler wird erkannt und in den Fehlermeldungen angezeigt	X
FT-16	Strukturfehler erkennen	Alle verschiedenen Strukturfehler senden	Alle Strukturfehler werden richtig erkannt, angezeigt und können angeklickt werden	X
FT-17	Linienfehler anzeigen	Maschine sendet drei verschiedene linieninterne Fehlermeldungen	Fehler und Sender werden erkannt, angezeigt und sind anklickbar	X

Tabelle B.1: Ergebnisse der durchgeführten Funktionstests

C Details zu Laboraufbauten und Messergebnissen

C.1 Versuchsanordnung

C.2 Liste der verwendeten Messgeräte

C.3 Übersicht der Messergebnisse

C.4 Schaltplan und Bild der Prototypenplatine

D Zusatzinformationen zu verwendeter Software

D.1 Struktogramm des Programmentwurfs

D.2 Wichtige Teile des Quellcodes

E Datenblätter

Auf den folgenden Seiten wird eine Möglichkeit gezeigt, wie aus einem anderen PDF-Dokument komplette Seiten übernommen werden können, z. B. zum Einbindungen von Datenblättern. Der Nachteil dieser Methode besteht darin, dass sämtliche Formateinstellungen (Kopfzeilen, Seitenzahlen, Ränder, etc.) auf diesen Seiten nicht angezeigt werden. Die Methode wird deshalb eher selten gewählt. Immerhin sorgt das Package „*pdfpages*“ für eine korrekte Seitenzahleinstellung auf den im Anschluss folgenden „nativen“ $\text{\LaTeX}$ -Seiten.

Eine bessere Alternative ist, einzelne Seiten mit „*\includegraphics*“ einzubinden.

Sachwortverzeichnis

Künstliche Intelligenz, 63